
LLM Agents: A Survey

Jungseob Lee

Korea University, South Korea

omanma1928@korea.ac.kr

ABSTRACT

Large language models have progressed from text predictors into the reasoning core of *autonomous agents*, systems that plan, invoke tools, maintain memory, and coordinate with other agents to pursue goals over long horizons. The resulting literature has grown explosively but unevenly, splitting into subcommunities that rarely cite one another. This survey imposes a *function-first* taxonomy that organizes the field by the architectural role each component plays rather than by the framework that introduced it: four component pillars (planning and reasoning, memory, tool use, and multi-agent coordination) grounded in two context dimensions (interactive environments and application domains) and assessed along two cross-cutting concerns, evaluation and safety, which we treat as first-class pillars rather than afterthoughts. Synthesizing more than two hundred works from 2021 through early 2026, including the reasoning-native models, computer-use agents, and agent-security research absent from earlier surveys, we find a field pulling in two directions: its components are consolidating, as prompting gives way to trained reasoning and scaffolds migrate into the models, while its frontier, grounding agents in real environments, evaluating them honestly for cost and reliability, and defending them against attack, remains wide open. Throughout, we give documented negative results on self-correction, multi-agent debate, and agentic scaffolding equal weight with positive claims, support the argument with small controlled experiments of our own, and show that capability and safety are distinct axes that must be measured apart. A continuously updated reading list of the surveyed papers is maintained at <https://github.com/js-lee-AI/awesome-llm-agent-papers>.

Keywords: large language models, autonomous agents, planning, memory, tool use, multi-agent systems, agent evaluation, agent safety, survey

Table of Contents

1	Introduction	4
2	Background: From Language Models to Agents	7
2.1	Language Models as Reasoning Cores	7
2.2	What Is an LLM-Based Agent?	7
2.3	A Cognitive-Architecture View	8
3	Taxonomy of LLM-Based Agents	9
4	Planning and Reasoning	9
4.1	Prompted Reasoning	11
4.2	Search over Thoughts	11
4.3	Task Decomposition and Interactive Planning	12
4.4	Reflection and Self-Correction	12
4.5	Learned Reasoning and Trained Agents	13
5	Memory	15
5.1	A Taxonomy of Agent Memory	15
5.2	Working Memory and Context Management	15
5.3	Long-Term and Structured External Memory	16
5.4	Experiential Learning and Skill Libraries	16
6	Tool Use and Action Execution	17
6.1	Foundations and Paradigms	17
6.2	Tool Selection, Creation, and Composition	18
6.3	Code as a Universal Action Space	18
6.4	Function-Calling Models and Training Data	18
7	Multi-Agent Systems	19
7.1	Role-Based Collaboration	20
7.2	Debate and Discussion	20
7.3	Scaling and Ensembling	21
7.4	Communication Topology and Team Optimization	21
8	Agents in Interactive Environments	22
8.1	Web Agents	22
8.2	GUI and Computer-Use Agents	23
8.3	Embodied and Robotic Agents	23
9	Evaluation and Benchmarks	24
9.1	What Is Hard about Evaluating Agents	24
9.2	The Benchmark Landscape	24
9.3	Methodological Critique	25
10	Applications	26
10.1	Software Engineering	26
10.2	Scientific Discovery	26
10.3	Healthcare	27

10.4 Finance	27
11 Safety, Security, and Trustworthiness	28
11.1 The Agentic Threat Model	28
11.2 Attacks on LLM-Based Agents	28
11.3 Risk Evaluation and Guardrails	29
11.4 Alignment-Level Risks	30
11.5 Autonomy Governance	31
12 Open Challenges and Future Directions	31
13 Conclusion	33

1 Introduction

Large language models (LLMs) have progressed from single-turn text predictors into the reasoning core of *autonomous agents*¹: systems that plan, invoke external tools, accumulate memory across steps, and coordinate with other agents to pursue goals over long horizons. Three base-model capabilities made this shift possible. Multi-step reasoning turned out to be elicitable by prompting alone (Wei et al. 2022), and even without exemplars (Kojima et al. 2022), so a model could be asked to deliberate before committing to an action. Instruction tuning made that deliberation steerable by ordinary task descriptions. And models proved able to operate interfaces they were never trained on, learning when and how to call external APIs from a handful of demonstrations (Schick et al. 2023). ReAct (Yao et al. 2022b) composed these capabilities into the loop that still defines the field: the model interleaves reasoning traces with actions and folds the resulting observations back into its next thought.

That loop escaped the laboratory almost immediately. Within months, AutoGPT-style systems chained it into open-ended autonomy (Yang et al. 2023b), and Generative Agents showed that populations of such loops, given memory and reflection, produce believable social behavior (Park et al. 2023a). Three years later, descendants of the same recipe resolve real GitHub issues, operate browsers and desktop interfaces on a user’s behalf, and design and run scientific experiments (§8, §10). Figure 2 traces this trajectory.

The literature that documents this progress has grown faster than its organization. Research on LLM-based agents is split across subcommunities, planning and reasoning, memory, tool learning, multi-agent systems, web and GUI automation, embodied control, agent safety, each with its own benchmarks, vocabulary, and citation graph, and with limited contact between them. The foundational general surveys unified the field as it stood in 2023 (Wang et al. 2023b; Xi et al. 2023), but predate three developments that have since reshaped it: reasoning-native models trained to deliberate, computer-use agents that act through raw screen interfaces, and the systematic study of agent security. More recent general surveys organize the field by methodology or framework lineage (Luo et al. 2025; Plaat et al. 2025; Masterman et al. 2024), by a brain-inspired decomposition into cognitive modules (Liu et al. 2025), or trace a single thread such as the road from chain-of-thought prompting to language agents (Zhang et al. 2023c). Across all of them, evaluation methodology and safety appear as closing discussions rather than as objects of analysis in their own right.

The scale of the problem is easy to understate. Figure 1 plots monthly arXiv submissions across five agent subtopics: of the roughly nine thousand papers our keyword harvest assigns to the field, fewer than one in three hundred predate 2023, yet the field now adds on the order of a thousand papers per month and its cumulative volume roughly tripled in the year to mid-2026. That growth is spread across planning, multi-agent, tool-use, computer-use, and safety subcommunities that expand in parallel and largely in isolation, exactly the fragmentation a function-first organization is meant to make navigable.

This survey organizes the field by *architectural function* instead. Frameworks bundle components and are superseded; the functions those components serve persist. We therefore structure the literature along three axes (Figure 4, developed in Section 3): four *component* pillars that describe what an agent is made of, planning (§4), memory (§5), tool use (§6), and multi-agent coordination (§7); two *context* dimensions that describe where it acts and where it is deployed, interactive environments (§8) and application domains (§10); and two *cross-cutting* concerns that apply to every cell of that matrix, evaluation (§9) and safety (§11). The function-first organization is what makes the fragmented subcommunities commensurable: a memory paper and a web-agent paper rarely cite each other, but both take a position on the same small set of design questions, and the taxonomy makes those positions comparable. Read through that lens, the field tells a single story: its *components* are consolidating as prompting gives way to trained reasoning and scaffolds migrate into the models; its *frontier* has moved to grounding agents

¹ **Agent.** A system that perceives an environment, maintains internal state, and selects actions to pursue a goal, typically in a closed loop (Sumers et al. 2023); formalized in §2.2.

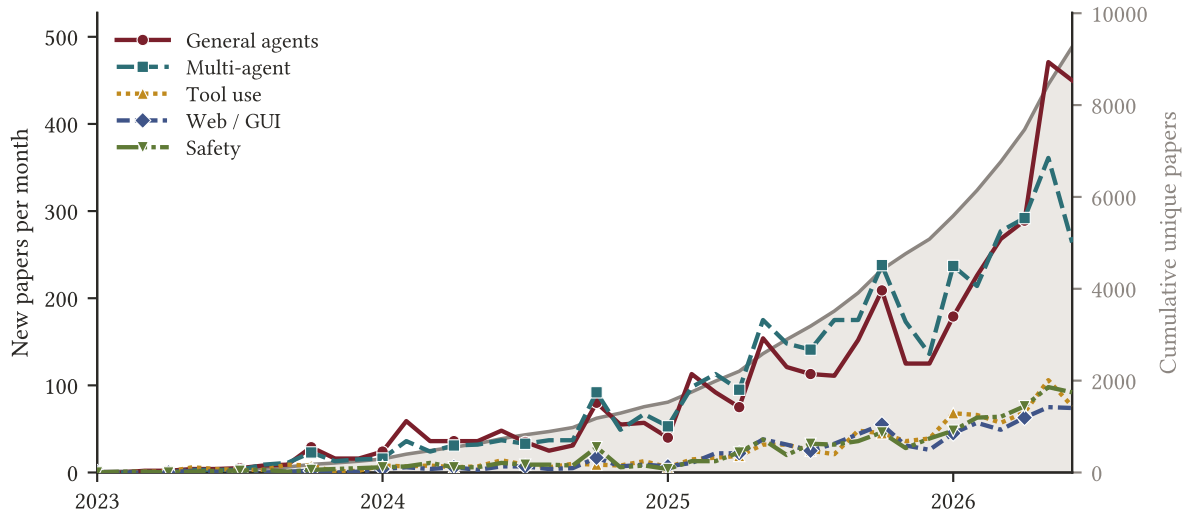


FIGURE 1: Growth of the LLM-agent literature. Monthly counts of new arXiv submissions in five agent subtopics (left axis) and the cumulative number of distinct papers (shaded, right axis), from January 2023 – when the literature begins in earnest, through mid-2026. Papers are matched by title and abstract keyword search and deduplicated by identifier for the cumulative count; keyword matching has high precision but imperfect recall, so the counts are conservative lower bounds.

in real environments and evaluating them honestly; and *safety* is the binding constraint on both. This is the thesis the rest of the survey defends.

Our contributions are as follows:

- **A function-first taxonomy** of LLM-based agents (four component pillars, two context dimensions, two cross-cutting concerns) that assigns every surveyed work a position independent of the framework that introduced it (Figure 4).
- **Current coverage.** We synthesize the literature from the first trained browsing agents (2021) through early 2026, including the post-2023 developments absent from the foundational surveys: reasoning-native models, computer-use agents, AI-scientist systems, and agent security.
- **Evaluation as a first-class pillar.** Beyond cataloguing benchmarks, Section 9 analyzes the validity of current evaluation practice, including cost-blind leaderboards, contamination, and outcome-versus-process metrics.
- **Safety and security as a first-class pillar.** Section 11 develops a unified agentic threat model spanning prompt injection, memory poisoning, and backdoors, through alignment-level risks and autonomy governance.
- **Original controlled experiments.** Rather than argue the survey’s two central trade-offs only from the literature, we measure them: a compute-matched comparison of agent control loops on ALFWorld (§4.3) and a capability-versus-injection-robustness measurement across model scale and family (§11.3).

Throughout, where the literature contains credible negative results, on self-correction (§4.4), on multi-agent debate (§7.2), and on agent scaffolds versus simple pipelines (§10.1), we present them with the same weight as the positive claims.

A note on method. The corpus behind this survey was assembled through ten subtopic sweeps (foundational surveys and position papers; single-agent architectures; planning and reasoning; memory; tool use; multi-agent systems; embodied, web, and GUI agents; benchmarks and evaluation methodology; safety and alignment; applied agents), yielding a core of 184 references, each verified against its primary source; further works were added under the same verification rule, for 228 references in total.

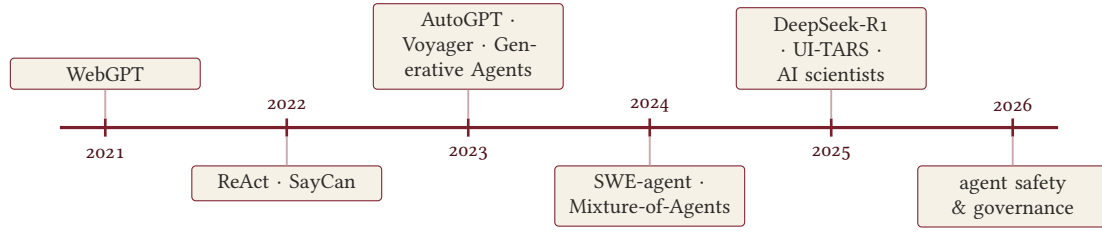


FIGURE 2: A timeline of representative milestones in LLM-based agents, from the first trained browsing agent (2021) to the reasoning-native, computer-use, and AI-scientist systems of 2025; the rightmost tick marks the present frontier of agent safety and governance (§11, §12) rather than a single dated system. Each system is introduced and cited in the section indicated by the taxonomy of Figure 4; labels alternate above and below the spine only for legibility.

TABLE 1: Comparison with prior general surveys of LLM-based agents. ✓ = an organizing pillar with dedicated sections; ~ = discussed, but not an organizing pillar; ✗ = absent or mentioned only in passing. “Coverage” is the latest literature year substantively covered.

Survey	Plan.	Mem.	Tools	Multi-ag.	Envs.	Eval. meth.	Safety	Coverage
Wang et al. (2023b)	✓	✓	~	~	~	~	✗	2023
Xi et al. (2023)	✓	~	✓	✓	~	~	~	2023
Zhang et al. (2023c)	✓	✗	~	✗	✗	~	✗	2023
Durante et al. (2024)	~	~	~	~	✓	✗	~	2024
Guo et al. (2024a)	~	~	~	✓	~	~	✗	2024
Masterman et al. (2024)	✓	~	✓	✓	✗	✗	~	2024
Liu et al. (2025)	✓	✓	~	✓	~	~	✓	2025
Luo et al. (2025)	✓	✓	✓	✓	~	~	~	2025
Plaat et al. (2025)	✓	~	✓	✓	~	✗	~	2025
This survey	✓	✓	✓	✓	✓	✓	✓	2026

We include a system if it places an LLM in an action–perception loop, the model’s outputs are executed against an environment, a tool interface, or another agent, and the results are fed back, and we exclude chat-only uses of LLMs, base-model training techniques except where they target agentic behavior, and vision–language–action robotics beyond what Section 8.3 requires.

Table 1 positions this survey against the closest general surveys. Single-pillar surveys, planning (Huang et al. 2024), memory (Zhang et al. 2024f), tool use (Qu et al. 2024), GUI agents (Nguyen et al. 2024), evaluation (Yehudai et al. 2025), and trustworthiness (Yu et al. 2025), go deeper than we do on their single column; we cite them as deeper dives in the corresponding sections and direct our own coverage at the connective tissue between pillars, which is precisely what a single-pillar survey cannot supply.

The remainder of the survey follows the taxonomy. Section 2 sets up definitions and the cognitive-architecture vocabulary; Section 3 presents the taxonomy itself. Sections 4–7 survey the four component pillars, Section 8 grounds them in web, GUI, and embodied environments, Section 9 reviews evaluation practice and its blind spots, Section 10 surveys deployed application domains, and Section 11 analyzes safety and security. Section 12 distills seven open challenges, and Section 13 concludes. Readers interested in a single pillar can read Sections 2–3 and jump directly to the relevant section; each pillar section is self-contained and closes with a key takeaway.

2 Background: From Language Models to Agents

This section establishes the minimal background the rest of the survey builds on: the base-model capabilities agents inherit (§2.1), a working definition of an LLM-based agent (§2.2), and the cognitive-architecture vocabulary we adopt throughout (§2.3).

2.1 Language Models as Reasoning Cores

Everything an agent can do is bounded by what its underlying model can do, so we begin with the base-model capabilities that agents inherit. Language modeling passed through several regimes, from count-based n -gram models, through distributed word representations and recurrent networks, to the pretrained Transformer that made transfer learning the default, but the capability that turned language models into agent controllers is a property of scale rather than of any single architecture. At sufficient scale, models acquire *in-context learning*: they infer a task from a few examples in the prompt, with no gradient updates. Instruction tuning then made this latent competence addressable through ordinary natural-language commands, and reinforcement learning from human feedback aligned model outputs with those commands; the same post-training machinery is what later transfers these capabilities across languages and domains (Lee et al. 2025).

The ability most directly responsible for agency is multi-step reasoning. Chain-of-thought prompting showed that eliciting intermediate reasoning steps sharply improves performance on arithmetic, commonsense, and symbolic tasks, and that this benefit emerges only past a scale threshold (Wei et al. 2022); the same behavior can be triggered zero-shot by a single instruction to reason step by step (Kojima et al. 2022). Because reasoning is latent and promptable, a model can be asked to deliberate before it acts, the precondition for treating the model as a decision-maker rather than a text generator. Early surveys mapped the space of reasoning-elicitation methods and their evaluation (Huang and Chang 2022), a space we revisit as agent planning in Section 4.

A further inflection, unfolding across 2024–2025, moved reasoning from a prompting trick into the model’s weights. Bootstrapping a model on its own correct rationales (Zelikman et al. 2022), rewarding individual reasoning steps rather than only final answers (Lightman et al. 2023), and, most strikingly, incentivizing reasoning through large-scale reinforcement learning alone (DeepSeek-AI et al. 2025) produced *reasoning-native* models that deliberate, verify, and backtrack without being prompted to. Frontier model reports now advertise agentic tool use and computer operation as headline capabilities (Comanici et al. 2025; Kimi Team 2025). These models unsettle a long-standing assumption of agent design (that planning must be supplied by an external scaffold) a tension we take up in §4.5.

2.2 What Is an LLM-Based Agent?

Definition 1 (LLM-Based Agent)

An *LLM-based agent* is a system in which an LLM serves as the controller of an action–perception loop: it receives an observation (task instruction, tool output, or message from another agent), updates an internal state (plan, memory, or scratchpad), and emits an action (a tool call, a message, or a final answer).

Definition 1 names four moving parts, each a pillar of this survey. The loop is closed by the model’s *decision procedure* (how it turns observations and internal state into the next action) which we survey as planning and reasoning (Section 4). The internal state that survives across steps is *memory* (Section 5). The external actions are, in the general case, *tool calls* (Section 6); when the recipient of an action is another agent, the action is a message, and the loop becomes a *multi-agent system* (Section 7). Figure 3 draws this anatomy; later sections refer to its blocks by name, and Section 11 re-annotates the same diagram to locate where attacks enter.

The perceive–decide–act loop is not new: it is the organizing abstraction of the belief–desire–intention (BDI) tradition and of reinforcement learning, both of which long predate LLMs. What the language model contributes is a general prior over world knowledge and, crucially, a *natural-language*

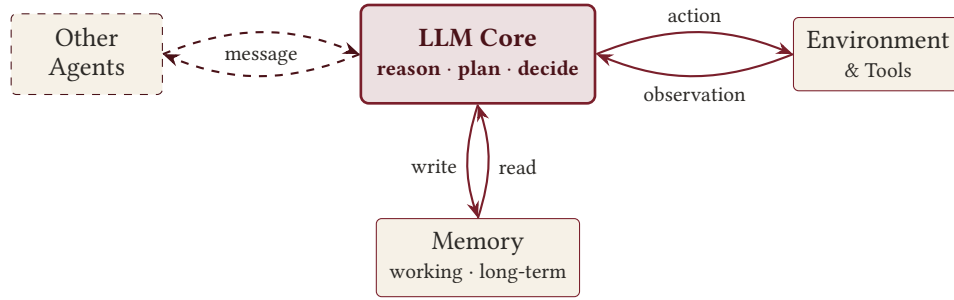


FIGURE 3: Anatomy of the LLM-agent loop. The LLM core maps observations and retrieved memory to actions; external actions act on an environment and its tools (solid), messages pass to and from other agents (dashed), and memory is read and written each step. Each block is the subject of one component pillar of this survey (§4–§7), and Figure 9 annotates the same diagram with attack entry points.

action interface. A classical agent must be given a hand-engineered state representation and, for RL, a task-specific reward; an LLM-based agent instead reads instructions, tool documentation, and observations as text and emits actions as text, so a single model generalizes across tasks it was never explicitly trained for. This is the capability that the earliest LLM-agent systems exploited, WebGPT learned to answer questions by operating a text browser under human feedback (Nakano et al. 2021); MRKL routed queries between the model and external symbolic modules (Karpas et al. 2022); and ReAct showed that merely interleaving reasoning with actions in the prompt yields a competent agent with no additional training (Yao et al. 2022b).

Terminology in this area is unsettled. We use *agent* for any system matching Definition 1; some authors reserve *agentic AI*² for higher-autonomy systems and contrast it with simpler tool-augmented models (Sapkota et al. 2025). Autonomy is best read as a spectrum rather than a binary, with proposed frameworks distinguishing several levels by how much of the loop the human retains (Feng et al. 2025b); whether systems at the top of that spectrum should be built at all is actively contested (Mitchell et al. 2025). We defer the governance of autonomy to §11.5 and use the levels here only as descriptive vocabulary.

2.3 A Cognitive-Architecture View

To describe agents precisely we adopt the vocabulary of Cognitive Architectures for Language Agents (CoALA) (Sumers et al. 2023), which recasts a diverse body of systems (ReAct, Reflexion, Voyager, and others surveyed below) in terms borrowed from classical cognitive architecture. CoALA organizes an agent along three axes. Its *memory* is divided into a short-lived working memory and long-term stores that, following human memory, are episodic (past experience), semantic (facts about the world), and procedural (skills and the agent’s own code). Its *action space* is split into *internal* actions that read from or write to those memories (reasoning, retrieval, learning) and *external* actions that ground the agent in an environment through tools or physical actuators. Its *decision procedure* is the loop that proposes, evaluates, and selects the next action.

These three axes are exactly the seams along which this survey cuts the literature. Planning (Section 4) is the decision procedure; memory (Section 5) is the store hierarchy; tool use (Section 6) is the external action space; and multi-agent coordination (Section 7) generalizes the action space to messages exchanged with other agents. Adopting a component vocabulary, rather than a per-framework one, is what lets us compare systems that were never designed to be compared: a framework such as AutoGen or MetaGPT bundles a particular choice along each axis, but the axes themselves are stable, and it is at the level of the axis that the design questions recur. Section 3 builds the survey’s taxonomy on this

² **Agentic AI.** An umbrella term, popularized in 2024–2025, for systems exhibiting autonomous multi-step goal pursuit; several authors use it to distinguish orchestrated multi-component systems from single tool-augmented models (Sapkota et al. 2025). We treat it as a synonym for the higher-autonomy region of Definition 1.

observation.

3 Taxonomy of LLM-Based Agents

The preceding section described one agent; this section organizes the thousands of them that the literature has produced. We classify the field along three kinds of axis. *Component* axes ask what an agent is made of, and following the cognitive-architecture view of §2.3 there are four: planning and reasoning (§4), memory (§5), tool use (§6), and multi-agent coordination (§7). *Context* axes ask where the agent acts: the interactive environment it is embedded in (§8) and the application domain it is deployed to (§10). *Cross-cutting* axes ask how we should judge it, and apply to every cell of the component-by-context matrix: evaluation methodology (§9) and safety (§11). Figure 4 lays out the full taxonomy; each leaf names a subsection of this survey together with representative systems, all cited in full where they appear.

The component pillars are the load-bearing distinction, because each is a mature subfield with its own dedicated survey that goes deeper than we can here: planning (Huang et al. 2024; Cao et al. 2025), memory (Zhang et al. 2024f; Wu et al. 2025), tool use (Qu et al. 2024; Wang et al. 2024g), and multi-agent systems (Guo et al. 2024a; Yan et al. 2025; Tran et al. 2025). We cite these as deeper dives at the head of each pillar and aim our own contribution at the connections between pillars, how a memory design constrains a planning strategy, how a tool interface reshapes an action space, which no single-pillar survey covers. The context axes are served by their own specialized surveys as well: web (Ning et al. 2025) and GUI agents (Nguyen et al. 2024; Zhang et al. 2024a) for environments, and domain surveys for software (Liu et al. 2024a), science (Ren et al. 2025), medicine (Wang et al. 2025c), and finance (Ding et al. 2024) for applications. The two cross-cutting concerns have recently acquired dedicated surveys of their own, evaluation (Yehudai et al. 2025; Mohammadi et al. 2025) and trustworthiness (Yu et al. 2025; Su et al. 2025), a sign that the field now treats them as objects of study rather than afterthoughts, and a stance this survey adopts by giving each a full section.

Why organize by function at all, rather than by framework, which is how practitioners often navigate the field? Because frameworks are bundles and functions are atoms. A system such as AutoGen (Wu et al. 2023a) or MetaGPT (Hong et al. 2024) fixes a particular choice on every axis at once (a role structure, a memory scheme, a tool protocol, a planning loop) and is then superseded by the next bundle. The design questions, however, recur unchanged across bundles: whether to plan ahead or interleave planning with acting, whether memory should be flat text or a structured graph, whether tools should be selected or authored, whether agents should cooperate or debate. Classifying by function makes these questions, and the competing answers to them, the unit of comparison, so that a lesson learned in one framework transfers to the next. Table 2 makes the payoff concrete: it places representative systems against every axis at once, so that designs the taxonomy tree of Figure 4 necessarily scatters into separate leaves can be read side by side. The remainder of the survey works through the taxonomy one axis at a time.

4 Planning and Reasoning

Planning is the agent’s decision procedure, the policy that turns the current observation and internal state into the next action. It is the axis on which the field has moved fastest, and it is instructive to organize the work not by chronology but by *where the improvement comes from*. Four sources recur: better prompting of a fixed model (§4.1), explicit search over candidate reasoning paths (§4.2), decomposition and interleaving of planning with acting (§4.3), and feedback in the form of self-correction (§4.4). A fifth source, training the model to reason directly (§4.5), has recently begun to absorb the other four, and we close the section by asking what that means for agent design (Huang et al. 2024; Cao et al. 2025).

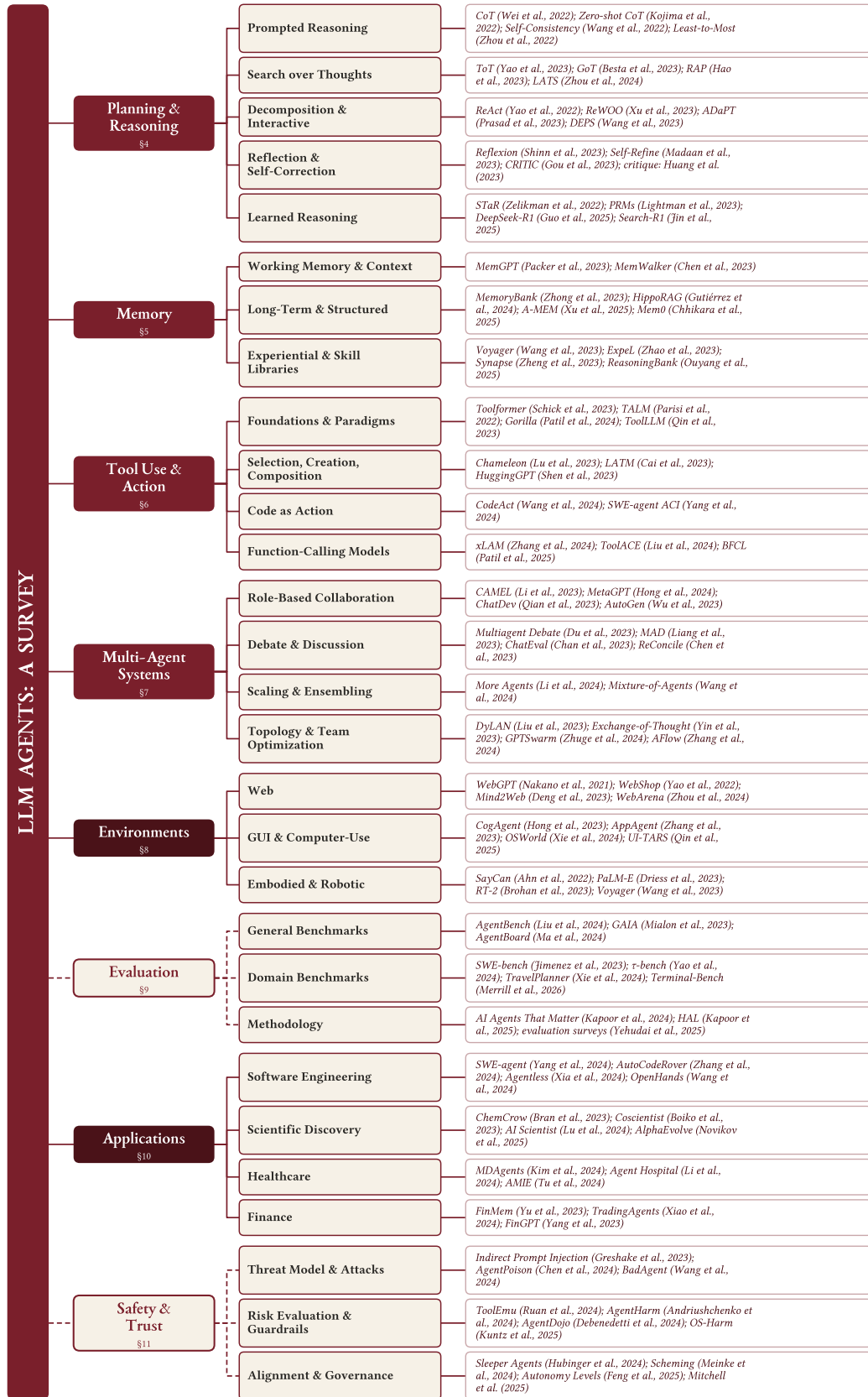


FIGURE 4: Master taxonomy of LLM-agent research as organized by this survey. Four *component* pillars (Planning & Reasoning §4, Memory §5, Tool Use §6, Multi-Agent Systems §7) are grounded in two *context* dimensions (Environments §8, Applications §10), with Evaluation (§9) and Safety (§11) as *cross-cutting* concerns (dashed edges). Leaves list representative works; all are cited in full in the corresponding sections.

TABLE 2: Representative agent systems placed against every axis of the taxonomy at once. “Coord.” is the multi-agent topology (“–” for single-agent); “Planning” names the decision procedure of §4. Every entry is documented in the section listed; this is the comparison the per-leaf tree of Figure 4 cannot show.

System	Planning	Memory	Action	Coord.	Domain / env.	§	Ref.
ReAct	interleaved	–	tools	–	general	4.3	(Yao et al. 2022b)
Reflexion	reflection	episodic	tools	–	general	4.4	(Shinn et al. 2023)
AutoGPT	plan-execute	scratchpad	tools	–	general	1	(Yang et al. 2023b)
HuggingGPT	plan-execute	–	models	–	multimodal	6.2	(Shen et al. 2023)
ToolLLM	DFS search	–	APIs	–	tool use	6.1	(Qin et al. 2023)
CodeAct	interleaved	–	code	–	general	6.3	(Wang et al. 2024e)
SWE-agent	interleaved	–	code (ACI)	–	software	6.3	(Yang et al. 2024a)
MemGPT	–	tiered	tool calls	–	dialogue	5.2	(Packer et al. 2023)
Voyager	curriculum	skill library	code	–	embodied	5.4	(Wang et al. 2023a)
CAMEL	role-play	dialogue	messages	dyad	general	7.1	(Li et al. 2023a)
AutoGen	configurable	per-agent	tools, code	star	general	7.1	(Wu et al. 2023a)
MetaGPT	SOP pipeline	shared docs	structured	pipeline	software	7.1	(Hong et al. 2024)
Gen. Agents	reflection	memory stream	messages	society	simulation	5.3, 7.4	(Park et al. 2023a)
WebVoyager	interleaved	–	GUI (pixels)	–	web	8.1	(He et al. 2024)
UI-TARS	trained eze	–	GUI actions	–	computer use	8.2	(Qin et al. 2025)
OS-Copilot	plan + improve	skill store	OS actions	–	OS	8.2	(Wu et al. 2024b)
ChemCrow	plan-execute	–	domain tools	–	chemistry	10.2	(Bran et al. 2023)
AI Scientist	plan-execute	–	code, tools	–	science	10.2	(Lu et al. 2024)

4.1 Prompted Reasoning

The cheapest way to make a model plan is to ask it to. Chain-of-thought prompting elicits intermediate reasoning steps that markedly improve multi-step problem solving (Wei et al. 2022), and the effect survives the removal of exemplars: a single instruction to reason step by step recovers much of the gain zero-shot (Kojima et al. 2022). These results reframed the model as something that can deliberate, and every planning method below is, in some sense, a more disciplined way of spending that deliberation.

The discipline takes three broad forms. Sampling many reasoning paths and taking a majority vote over their answers (self-consistency) trades inference compute for accuracy and reliably beats greedy decoding (Wang et al. 2022). Decomposing a problem into an ordered list of easier subproblems, each conditioned on the last, improves compositional generalization (Zhou et al. 2022). And splitting the work into an explicit plan-then-execute pair reduces the calculation and missing-step errors of naive zero-shot reasoning (Wang et al. 2023c). A theme already visible here, and central to Section 9, is that these gains are bought with extra inference: self-consistency’s accuracy scales with the number of sampled paths, so any comparison that ignores compute overstates the method’s value.

4.2 Search over Thoughts

If prompted reasoning explores a single path, the natural generalization is to search over many. Tree of Thoughts makes the search explicit, maintaining a tree of partial solutions that the model expands, evaluates, and backtracks over using breadth- or depth-first search (Yao et al. 2023). Graph of Thoughts relaxes the tree to an arbitrary graph so that reasoning paths can be merged and refined, not just branched (Besta et al. 2023). The move from chain to tree to graph (Figure 5) buys quality on hard problems at a steep and often unreported compute cost.³

A second line makes the search *model-based*. Reasoning via Planning repurposes the LLM as its own world model⁴, predicting the next state so that Monte Carlo Tree Search can plan against it (Hao et al. 2023a). Language Agent Tree Search extends this to grounded settings, unifying reasoning, acting,

³ Tree and graph search multiply token consumption by the branching factor and depth; the original papers report the accuracy gains but comparisons across methods rarely fix a compute budget, a gap Section 9.3 returns to. The underlying subword tokenizer further sets the per-step token budget (Lee et al. 2024).

⁴ **World model.** An internal predictor of how the environment (or the reasoning state) will change in response to an action; classical planning searches against such a model. In LLM agents the model is often the LLM itself, prompted to predict the next state (Hao et al. 2023a).

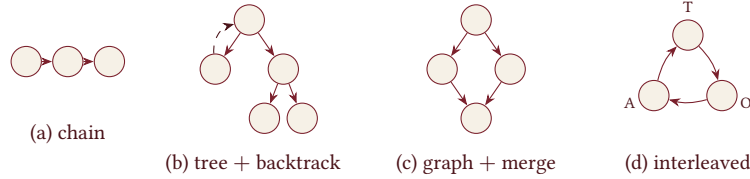


FIGURE 5: Four planning paradigms. (a) A linear chain of thoughts (§4.1); (b) a tree with self-evaluation and backtracking (Tree of Thoughts); (c) a graph that merges reasoning paths (Graph of Thoughts); (d) an interleaved loop that alternates Thought, Action, and Observation (ReAct-style, §4.3). Expressiveness rises left to right, and so does inference cost.

and reflection in a single search over an environment (Zhou et al. 2024a). A third, meta-level line has the model search over reasoning *strategies* rather than reasoning steps: Self-Discover composes task-specific reasoning structures from atomic modules (Zhou et al. 2024b), while Buffer of Thoughts retrieves distilled “thought-templates” from past problems (Yang et al. 2024b), a design that already blurs into the memory mechanisms of Section 5.

4.3 Task Decomposition and Interactive Planning

Search assumes the agent can evaluate a plan before executing it. In interactive settings it often cannot, because the environment reveals information only when acted upon, which raises the defining question of this subsection: should the agent commit to a full plan up front, or interleave planning with acting? ReAct takes the interleaved extreme, alternating a reasoning trace with an action and folding each observation back into the next thought (Yao et al. 2022b). ReWOO takes the plan-ahead extreme, producing a complete tool-use plan before any execution; by decoupling reasoning from observation it cuts token consumption roughly fivefold relative to interleaved loops, at the cost of being unable to adapt when an early step surprises it (Xu et al. 2023).

The productive answer is that decomposition granularity should be conditioned on the agent’s own competence rather than fixed in advance. ADaPT recurses into finer subtasks only when the executor fails to complete a step directly, matching plan depth to difficulty (Prasad et al. 2023). DEPS closes the loop differently, having the agent describe its progress, explain failures, and revise the plan, which lets a single zero-shot agent complete dozens of long-horizon tasks in an open world (Wang et al. 2023f). Both point the same way: the plan is not an artifact produced once but a hypothesis revised against feedback.

The trade-off this subsection describes is usually argued from the literature; we can also measure it. Figure 6 reports a small controlled comparison we ran of the same model (Qwen2.5-7B-Instruct) under three control loops on 50 ALFWorld tasks. Interleaving (ReAct) roughly doubles the success rate of plan-then-execute (ReWOO), 46% versus 22%, but costs about four times as many tokens; adding a single reflection retry (Reflexion, §4.4) buys a further 8 points to 54% at another 70% more tokens. The plan-ahead loop is genuinely cheaper (it just spends its savings on failures it cannot see coming) and the result lands squarely on a cost-accuracy frontier, foreshadowing the argument of Section 9.3 that neither axis should be read alone.

4.4 Reflection and Self-Correction

Revising against feedback invites an appealing idea: let the agent generate that feedback itself. Reflexion has the agent write a verbal self-reflection after a failed attempt and store it in episodic memory, improving decision-making on later attempts without any weight update (Shinn et al. 2023). Self-Refine applies the same loop within a single problem, alternating generation and self-critique (Madaan et al. 2023), and CRITIC grounds the critique in external tools such as search engines and interpreters (Gou et al. 2023). Reported gains across these systems made self-correction a near-default component of agent scaffolds.

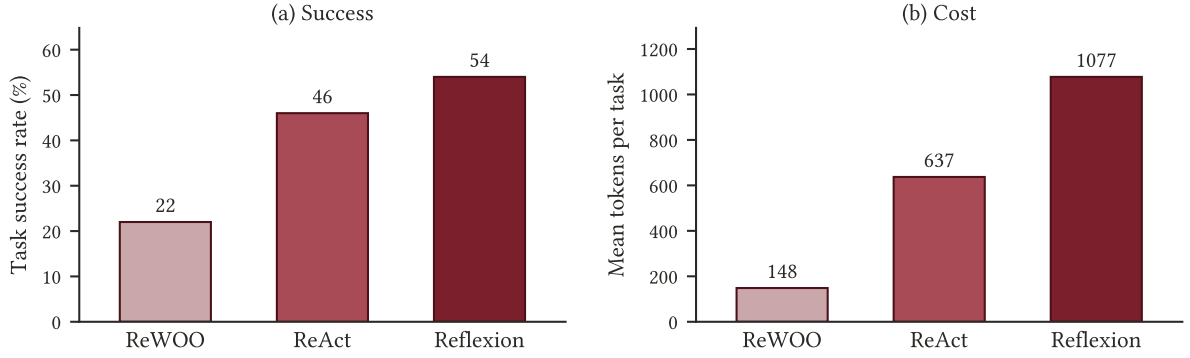


FIGURE 6: Cost and accuracy of three agent control loops sharing a single model (Qwen2.5-7B-Instruct) on 50 ALFWorld tasks. Plan-then-execute (ReWOO) is the cheapest but least successful; interleaving reasoning with acting (ReAct) and adding one reflection retry (Reflexion) raise the task success rate (a) at a steep cost in generated tokens (b). Success and cost rise together, so neither reading alone is complete.

TABLE 3: Planning strategies by the structure they search and the feedback they use. “Cost” is inference overhead relative to a single chain-of-thought pass (qualitative; exact factors depend on branching and depth). “Env?” marks whether the method requires interacting with an external environment or tools, as opposed to pure internal reasoning.

Strategy	Search structure	Feedback source	Cost	Env?	Ref.
Chain-of-Thought	linear chain	none	base	✗	(Wei et al. 2022)
Self-Consistency	parallel samples	answer vote	$N \times$	✗	(Wang et al. 2022)
Tree of Thoughts	tree	self-evaluation	high	✗	(Yao et al. 2023)
Graph of Thoughts	graph (merges)	self-evaluation	high	✗	(Besta et al. 2023)
RAP	tree (MCTS)	world-model value	high	✗	(Hao et al. 2023a)
LATS	tree (MCTS)	env. + reflection	high	✓	(Zhou et al. 2024a)
ReWOO	plan-then-act	tool observations	low	✓	(Xu et al. 2023)
Reflexion	linear + memory	env. reward	medium	✓	(Shinn et al. 2023)

The evidence for *intrinsic* self-correction, however, is weaker than the enthusiasm suggests. Controlled experiments find that when a model is asked to correct its reasoning using only its own judgment, with no external signal, performance frequently fails to improve and sometimes degrades, because the model cannot reliably tell which of its answers is wrong (Huang et al. 2023a). A critical survey of the subfield goes further, arguing that many positive results rest on ill-posed setups, oracle stopping criteria, unfair baselines, or tasks where the correction step smuggles in information the first attempt lacked (Kamoi et al. 2024). The two literatures are not actually in contradiction once the confound is named.

The reconciliation is that self-correction works when the feedback is *not* purely intrinsic. CRITIC’s tool grounding and Reflexion’s environment signal both supply an external check, which is why they help where pure self-critique does not. Where such a signal is unavailable, the alternative is to train the ability in: SCoRe uses multi-turn reinforcement learning on the model’s own correction traces to produce genuine self-correction, overcoming the distribution-shift and collapse failures of prompting-only approaches (Kumar et al. 2024). Self-correction, in short, is real but not free, it requires either an external verifier or a training signal, a conclusion that anticipates Section 4.5 and the section’s key takeaway.

4.5 Learned Reasoning and Trained Agents

The methods above all treat the model as fixed and wrap machinery around it. The most recent shift instead trains the model to reason. STaR bootstraps a model on the rationales that lead to correct answers, iteratively improving its reasoning from a small seed (Zelikman et al. 2022). Process supervision

TABLE 4: From prompted to trained agents. Methods are ordered from training static reasoning to training agents online in an environment. “Env?” marks whether training uses a live interactive environment (✓), only offline traces recorded from one (∼), or no environment (✗).

Method	Training signal	Data source	Env?	Target	Ref.
STaR	rationale bootstrap (SFT)	self-generated	✗	reasoning	(Zelikman et al. 2022)
DeepSeek-R1	outcome RL	self-generated	✗	reasoning	(DeepSeek-AI et al. 2025)
SCoRe	multi-turn RL	self-generated	✗	self-correction	(Kumar et al. 2024)
AgentTuning	trajectory SFT	teacher traces	∼	general agent	(Zeng et al. 2023)
FireAct	trajectory SFT	teacher ReAct traces	∼	reasoning + tools	(Chen et al. 2023a)
xLAM	function-call SFT	synthetic pipeline	✗	tool calling	(Zhang et al. 2024c)
ToolACE	function-call SFT	verified synthetic	✗	tool calling	(Liu et al. 2024b)
Search-R1	outcome RL	self-gen. + live search	✓	reasoning + search	(Jin et al. 2025)
ReTool	outcome RL	self-gen. + code exec.	✓	reasoning + code	(Feng et al. 2025a)
WebRL	online curriculum RL	self-evolving	✓	web	(Qi et al. 2024)
AgentGym	SFT + online RL	many environments	✓	general agent	(Xi et al. 2024)
UI-TARS	SFT + RL	GUI trajectories	✓	computer use	(Qin et al. 2025)

rewards each intermediate step rather than only the final answer, and outperforms outcome supervision on hard mathematical reasoning (Lightman et al. 2023). DeepSeek-R1 pushes the idea to its limit, showing that large-scale reinforcement learning with only an outcome signal can induce reasoning-native behavior (self-verification, backtracking, and long deliberation) as an emergent property, with no supervised reasoning traces at all (DeepSeek-AI et al. 2025).

Training has since moved from teaching a model to reason to teaching it to *act*, and this is the concrete mechanism behind the survey’s recurring observation that scaffolds are migrating into the models. A first wave fine-tunes on agent trajectories: AgentTuning mixes interaction traces into instruction tuning to give general models agent abilities (Zeng et al. 2023), and FireAct fine-tunes on ReAct-style trajectories drawn from several tasks (Chen et al. 2023a). A second wave trains agents *online*, in the environment itself: WebRL builds a self-evolving curriculum with reinforcement learning for web agents (Qi et al. 2024), and AgentGym evolves agents across many interactive environments at once (Xi et al. 2024). A third wave moves the tools inside the reasoning chain itself: Search-R1 trains a model to interleave search-engine calls with its own thoughts from outcome rewards alone (Jin et al. 2025), and ReTool learns when to invoke a code interpreter mid-derivation (Feng et al. 2025a); this agentic reinforcement learning is growing fast enough to have a survey of its own (Zhang et al. 2025). The same move recurs in the capability-specific models surveyed later, function-calling models trained on synthetic call data (§6.4), reinforcement-learned GUI agents such as UI-TARS (§8.2), and RL-trained self-correction (SCoRe, §4.4). Table 4 organizes these by the training signal they use and by whether that signal is drawn from a live environment, which is the axis along which prompted scaffolds are being absorbed into trained behavior.

This raises a genuinely open question for agent design, and we flag it as open rather than resolve it. If a model backtracks and verifies internally, do the external scaffolds of §4.2–§4.4 (the search trees, the reflection loops) remain necessary, or does planning migrate into the weights and leave the scaffold redundant on the tasks the model has internalized? Early evidence is mixed, and the pillar surveys of learned planning treat it as unsettled (Huang et al. 2024; Cao et al. 2025); we return to it as a future direction in Section 12.

Key Takeaway

Across prompting, search, decomposition, reflection, and training, the gains in agent planning come from better *verification and selection* of candidate reasoning, not from more fluent generation. Verification (an external check, a learned value, or a trained-in signal) is the bottleneck, and methods that lack it (pure self-correction, compute-blind search) do not reliably improve.

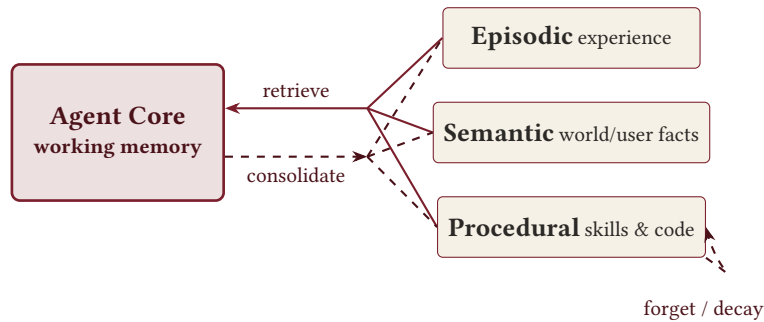


FIGURE 7: The design space of agent memory. Long-term stores are read by a *retrieval* policy (solid) and written by a *consolidation* policy (dashed), with an optional *forgetting* policy that decays stale entries. Systems differ mainly in the store form and in these three policies (Table 5), not in raw capacity.

5 Memory

If planning is what an agent does within a step, memory is what carries across steps. A language model is, by default, stateless: it retains nothing between calls beyond what fits in its context window. Memory is the machinery that lifts an agent out of that amnesia, it is what lets an agent learn a user’s preferences over a month, reuse a skill it acquired last week, or avoid a mistake it made an hour ago, and it is, in the strong sense, the component that makes an agent more than a prompt.

5.1 A Taxonomy of Agent Memory

Following the cognitive-architecture view of §2.3, agent memory divides first by time horizon and then by content. By horizon there is *working memory*, the information immediately available in the context window during a single decision, and *long-term memory*, which persists across episodes. Long-term memory, in turn, borrows the human tripartite division (Sumers et al. 2023; Wu et al. 2025): *episodic* memory of past experiences and interactions, *semantic* memory of facts about the world and the user, and *procedural* memory of skills and routines the agent can execute. The pillar survey of Zhang et al. (2024f) organizes the subfield along compatible lines and provides a fuller catalogue than we give here.

This four-way split is descriptive; what makes memory systems differ in practice is a small set of design decisions, and it is these that structure the rest of the section (Figure 7). Every memory system must choose a *store form* (flat text, vectors, key–value pairs, or a graph), a *read policy* (how relevant memories are retrieved for the current step), a *write policy* (what gets committed to long-term storage and in what form), and a *forgetting policy* (how stale or low-value memories are decayed, if at all). The recurring finding of the subfield is that these policies, not raw capacity, determine whether memory helps.

5.2 Working Memory and Context Management

The first problem memory poses is that working memory is bounded: the context window is finite, yet tasks are not. MemGPT reframes this as an operating-systems problem, giving the agent a tiered memory hierarchy and letting it *page* information between a limited main context and external storage through self-issued function calls, much as an OS moves data between RAM and disk (Packer et al. 2023). MemWalker takes a complementary route for long documents, building a tree of recursively summarized nodes and having the agent navigate it top-down at query time, reading only the branches it needs (Chen et al. 2023b). Both make the same conceptual move: context management is not a fixed preprocessing step but an *action* the agent takes, subject to the same decision-making the rest of the loop uses.

5.3 Long-Term and Structured External Memory

Long-term memory systems are best compared by their store form, because form dictates what the read and write policies can be. The simplest form is flat text with a lifecycle. The canonical design is the *memory stream* of Generative Agents (Park et al. 2023a): a chronological log of natural-language observations retrieved by a score combining recency, importance, and relevance, with a periodic reflection step that synthesizes low-level memories into higher-level ones, a template that most later text-memory systems vary. MemoryBank stores past interactions as text and adds a psychologically motivated forgetting rule, strengthening or decaying each memory by an Ebbinghaus curve so that importance and recency govern what survives (Zhong et al. 2023). Think-in-Memory refines what is stored, keeping the agent’s evolved *thoughts* rather than raw dialogue and updating them with insert, forget, and merge operations, which avoids the inconsistent re-reasoning that raw-text recall invites (Liu et al. 2023a).

Structuring the store buys sharper retrieval. RET-LLM records knowledge as subject–relation–object triplets, giving an updatable and interpretable memory (Modarressi et al. 2023); HippoRAG goes further, building an open knowledge graph and retrieving over it with Personalized PageRank in a deliberate analogy to hippocampal indexing, which enables single-step multi-hop recall that flat retrieval handles poorly (Gutiérrez et al. 2024). That store form matters is not merely asserted: a controlled study varying chunk-, triple-, fact-, and summary-based structures over identical queries finds the best structure depends on the query type, offering design guidance rather than a single winner (Zeng et al. 2024).

The newest systems make the store self-organizing or production-hardened. A-MEM links each new memory to related ones in a Zettelkasten-style network and lets new entries revise the attributes of old ones, so the memory continually restructures itself (Xu et al. 2025). Memo targets deployment, dynamically extracting and consolidating salient facts across sessions and reporting, on the LoCoMo long-conversation benchmark, large quality gains over a strong memory baseline, together with roughly ninety-percent reductions in latency and token cost relative to full-context prompting, a reminder that in production the binding constraint on memory is as much cost as accuracy (Chhikara et al. 2025). MIRIX pushes toward the frontier of multiple coordinated memory types, including multimodal experience, managed by dedicated per-type managers (Wang and Chen 2025). A separate, architecture-level line embeds memory in the model itself rather than in a scaffold: LongMem attaches a trainable side-network that caches and retrieves past context (Wang et al. 2023d), and Larimar equips an LLM with an episodic memory supporting fast one-shot updates and selective forgetting without retraining (Das et al. 2024).

Because these systems make competing claims, they need a common yardstick. LoCoMo supplies one, evaluating very long-term conversational memory with multi-hop and temporal questions that expose the weaknesses of both long-context and naive retrieval baselines (Maharana et al. 2024); it is now the standard benchmark by which the systems above report, with LongMemEval complementing it by decomposing long-term interactive memory into five separately testable abilities (Wu et al. 2024a). We return to memory evaluation in Section 9. Table 5 summarizes the design choices across representative systems.

5.4 Experiential Learning and Skill Libraries

Procedural memory closes the loop between memory and self-improvement. When an agent stores not facts but *skills*, memory becomes the substrate on which it gets better at its job without any weight update. Voyager makes this concrete in an open-ended embodied setting, maintaining a library of executable code skills that it writes, verifies, and later retrieves and composes, so that competence accumulates over a lifetime of play (Wang et al. 2023a). Synapse stores whole past trajectories as exemplars and retrieves them by task similarity to guide new episodes (Zheng et al. 2023).

Storing raw experience, however, scales poorly; the more useful object is often a distilled lesson. ExpeL has an agent gather experiences across training tasks and extract natural-language insights from

TABLE 5: Long-term memory systems by their store form and policies. “Open” marks a public implementation; “–” means not clearly available. Skill-library systems (Voyager, ExpeL) store procedural memory and are discussed in §5.4.

System	Store form	Read policy	Write policy	Forget	Open	Ref.
Gen. Agents	memory stream	recency/import./relev.	append + reflection	recency	✓	(Park et al. 2023a)
MemGPT	tiered context	self-issued paging	self-managed	evict	✓	(Packer et al. 2023)
MemoryBank	flat text	relevance retrieval	append + update	Ebbinghaus	✓	(Zhong et al. 2023)
HippoRAG	knowledge graph	Personalized PageRank	index into graph	none	✓	(Gutiérrez et al. 2024)
A-MEM	linked notes	similarity + links	note link + evolve	implicit	✓	(Xu et al. 2025)
Memo	text + graph	salient-fact retrieval	extract + consolidate	salience	✓	(Chhikara et al. 2025)
Larimar	episodic matrix	addressable read	one-shot update	selective	–	(Das et al. 2024)
Voyager	skill library	task-embedding search	add verified skill	none	✓	(Wang et al. 2023a)
ExpeL	insight rules	relevance retrieval	distill insights	none	✓	(Zhao et al. 2023)
ReasoningBank	strategy store	relevance retrieval	distill strategies	none	–	(Ouyang et al. 2025)

both its successes and its failures, carrying forward compact rules rather than full episodes and improving purely through in-context use of that memory (Zhao et al. 2023). ReasoningBank scales the same idea toward deployment, distilling strategy-level memory from both successful and failed trajectories so that an agent self-evolves over a continuous task stream (Ouyang et al. 2025). This is continual learning without gradients, and it is one of the clearest bridges from memory to the open problem of lifelong agent improvement we take up in Section 12.

Key Takeaway

What bounds an agent’s memory is not how much it can store but how well it can *read and write*: retrieval precision, consolidation policy, and principled forgetting distinguish the systems that help from those that merely accumulate. Structured stores and distilled experience outperform larger flat ones because they make retrieval and reuse tractable.

6 Tool Use and Action Execution

Memory lets an agent look inward across time; tools let it reach outward into the world. A tool call is the external action of Definition 1, the means by which an agent computes what its weights cannot, retrieves what they do not contain, and effects change beyond generating text. Tool use is therefore the pillar that turns a reasoner into an actor, and this section follows its arc from prompting a model to call fixed tools, through learning to call thousands of them, to letting the agent write executable code as its action space, and finally to training models specialized for the task.

Definition 2 (Tool)

A *tool* is an external program the agent invokes by emitting a structured call (an API, a function, a code interpreter, a search engine, or another model) whose execution returns an observation the agent reads back (Wang et al. 2024g). *Tool use* is any mechanism by which the agent selects, parameterizes, and sequences such calls.

6.1 Foundations and Paradigms

The literature splits along one axis: whether tool competence is *prompted* into a fixed model or *learned* into its weights. The learned branch came first at scale. TALM bootstrapped tool use from a few demonstrations through iterative self-play (Parisi et al. 2022), and Toolformer made the idea self-supervised,

letting a model annotate ordinary text with the API calls that would have helped predict it, then training on the calls that lowered loss (Schick et al. 2023). Scaling the tool inventory exposed a retrieval problem: Gorilla pairs a fine-tuned model with a document retriever so it can emit correct calls across thousands of constantly changing APIs without hallucinating arguments (Patil et al. 2024), and Tool-LLM extends this to over sixteen thousand real-world APIs, using a depth-first decision-tree search to plan and recover across multi-call traces (Qin et al. 2023).

The prompted branch asks how far a fixed model can be pushed with the right context, and the surprising answer is: quite far. Tool documentation alone, with no demonstrations, is often sufficient for zero-shot tool use and scales better than few-shot prompting when hundreds of tools are available (Hsieh et al. 2023). The two branches trade off predictably, prompted tool use scales with the size of the tool catalogue that fits in context, learned tool use scales with the volume of training data, a tension the pillar survey of Qu et al. (2024) maps in full and that the function-calling models of §6.4 attempt to resolve.

6.2 Tool Selection, Creation, and Composition

Calling one tool is selection; calling several in concert is composition, and it is where agents begin to resemble programs. Chameleon has an LLM synthesize a program that composes vision models, search, and Python functions to solve multimodal tasks (Lu et al. 2023); HuggingGPT generalizes the pattern by treating other models as the tools, using the LLM as a controller that plans a task and dispatches specialized models to execute its stages (Shen et al. 2023); and RestGPT drives real, stateful REST APIs through a coarse-to-fine planner and a dedicated executor that formats parameters and parses responses (Song et al. 2023).

More striking than composing given tools is authoring new ones. LATM closes a loop in which a capable model acts as a *tool maker*, writing reusable Python functions that a cheaper model then applies, amortizing the authoring cost across many uses (Cai et al. 2023). ToolkenGPT attacks the scaling limit from the other side, representing each tool as a learned vocabulary token (a “toolken”) so that a frozen model can select from a large, expandable tool set without listing every tool in its prompt (Hao et al. 2023b). Together these mark a shift in what the tool set is: no longer a fixed external inventory but a learned, self-extending object the agent grows.

6.3 Code as a Universal Action Space

Taken to its conclusion, the composition-and-authoring trend collapses the distinction between tool use and programming. CodeAct consolidates an agent’s entire action space into executable Python: instead of emitting a fixed JSON action schema, the agent writes code, which subsumes tool calls, control flow, and data manipulation in one interface and lets the agent revise from execution errors by debugging its own program (Wang et al. 2024e). Code as action is expressive precisely because it is Turing-complete, but realizing that expressiveness depends on the surrounding interface. SWE-agent makes this explicit with the notion of an agent–computer interface⁵: a deliberately designed set of commands and observations for browsing, editing, and running code, whose design changes task success as much as swapping the underlying model does (Yang et al. 2024a). We treat the ACI concept here and defer its software-engineering payoff to §10.1.

6.4 Function-Calling Models and Training Data

The prompted-versus-learned tension resolves, in practice, into a class of models trained specifically to call functions. xLAM is a family of “large action models” trained through a unified data-synthesis pipeline to strengthen tool use, topping function-calling leaderboards (Zhang et al. 2024c); ToolACE

⁵ **Agent–computer interface (ACI).** The commands and feedback formats through which an agent perceives and acts on a computer, the agent-facing analogue of a human user interface. SWE-agent shows that ACI design changes capability as much as the model does (Yang et al. 2024a).

TABLE 6: Representative tool-use benchmarks. “Execution” indicates whether tool calls are run against real APIs, a simulated/virtual server, or matched structurally; “Turns” distinguishes single-call from multi-turn tasks. Cross-referenced from §9.

Benchmark	Tools / APIs	Execution	Turns	Metric	Ref.
API-Bank	73 APIs	real (runnable)	multi	call correctness	(Li et al. 2023b)
StableToolBench	16k+ (virtual)	simulated server	multi	pass / win rate	(Guo et al. 2024b)
BFCL	multi-language	AST matching	single + multi	AST accuracy	(Patil et al. 2025)
GTA	real deployed tools	real execution	multi	task success	(Wang et al. 2024a)

reaches comparable accuracy with an 8B model by generating a diverse, rule- and model-verified corpus of function-calling dialogues (Liu et al. 2024b). Both sit in the trained-agent lineage organized in Table 4. Both are also downstream of a data problem, and the open-model lineage that precedes them is a history of solving it: GPT4Tools self-instructs a tool-use dataset from a teacher model (Yang et al. 2023d), ToolAlpaca synthesizes several thousand tool-use cases from a multi-agent simulation to fine-tune compact models (Tang et al. 2023), and a careful study shows small models are weak tool learners unless the task is decomposed across specialized planner, caller, and summarizer sub-models (Shen et al. 2024). Progress on function calling has been, more than anything, progress on generating verified training data.

These benchmarks, API-Bank’s runnable multi-turn dialogues (Li et al. 2023b), StableToolBench’s virtual API server that stabilizes the volatile ToolBench APIs (Guo et al. 2024b), the Berkeley Function-Calling Leaderboard’s abstract-syntax-tree matching (Patil et al. 2025), and GTA’s real-tool tasks with implicit intent on which leading models finish under half (Wang et al. 2024a), are summarized in Table 6 and revisited in Section 9.

The final step of this arc is standardization. As tool use matured, the per-model function-calling conventions began to converge on open protocols that decouple an agent from the tools it calls: the Model Context Protocol exposes tools and data to any client through a common interface (Anthropic 2024b), and the Agent2Agent protocol standardizes how agents discover and delegate to one another (Google 2025). These protocols make tools and agents composable across vendors, but by turning the tool interface into shared infrastructure they also enlarge the supply-chain attack surface of §11 and raise the open infrastructure questions of §12. The protocol layer is young but already an object of study: a survey of agent protocols maps the emerging standards and their evaluation dimensions (Yang et al. 2025), and a security analysis of the MCP ecosystem catalogues threats across the server lifecycle (Hou et al. 2025).

Key Takeaway

Tool use has progressed from prompting a model to call a fixed API, through learning to select among thousands, to writing executable code as a universal action space. Each step widens the action space and, with it, moves the dominant failure mode from *selection* errors (calling the wrong tool) toward *execution* errors (calling it wrongly), which is why verified execution feedback, not larger tool catalogues, drives recent gains.

7 Multi-Agent Systems

The three preceding pillars describe a single agent. When the recipient of an action is another agent rather than a tool or an environment, the loop of Figure 3 closes between agents, and coordination becomes a design problem in its own right. The motivation is partly a division of labor (specialized roles, separate contexts, parallel work) and partly a hope that agents checking one another will be more

TABLE 7: Representative multi-agent frameworks. “Roles” distinguishes fixed (assigned in advance) from dynamic (recruited or pruned at run time); “Human” marks support for a human in the loop.

Framework	Topology	Roles	Medium	Human	Domain	Ref.
CAMEL	dyad	fixed	natural language	✗	general	(Li et al. 2023a)
AutoGen	configurable	fixed	language + code	✓	general	(Wu et al. 2023a)
MetaGPT	pipeline (SOP)	fixed	structured docs	✗	software	(Hong et al. 2024)
ChatDev	pipeline (chain)	fixed	natural language	✗	software	(Qian et al. 2023)
AgentVerse	dynamic group	dynamic	natural language	✗	general	(Chen et al. 2023d)
MoA	layered	fixed	responses	✗	general	(Wang et al. 2024b)
DyLAN	dynamic network	dynamic	natural language	✗	reasoning	(Liu et al. 2023b)
Debate	mesh	symmetric	natural language	✗	reasoning	(Du et al. 2023)
GPTSwarm	learned graph	learned	messages	✗	general	(Zhuge et al. 2024a)
AFLOW	searched workflow	fixed	code	✗	general	(Zhang et al. 2024d)

reliable than any one alone. Whether that hope is borne out is the central, and contested, question of this section. Three pillar surveys map the space: a general survey of progress and challenges (Guo et al. 2024a), a communication-centric survey (Yan et al. 2025), and a survey of collaboration mechanisms (Tran et al. 2025).

Definition 3 (Multi-Agent System)

A *multi-agent system* (MAS) of LLM-based agents is a collection of agents, each matching Definition 1, that exchange messages according to a communication *topology* and a coordination *protocol* in order to solve a shared task. Agents may differ in role, tools, memory, persona, or underlying model.

7.1 Role-Based Collaboration

The earliest and most influential multi-agent designs assign agents *roles*. CAMEL showed that two agents (an instruction-giving user and a solving assistant) kept on task by “inception” prompts can cooperate autonomously, and used the setup to study emergent cooperative behavior (Li et al. 2023a). Giving roles a workflow made them productive: MetaGPT encodes human standard operating procedures into a structured assembly line of specialist agents, which curbs the error cascades of naive agent chaining (Hong et al. 2024), and ChatDev instantiates a virtual software company whose agents communicate along a structured chat chain (Qian et al. 2023). Generalizing beyond fixed pipelines, AutoGen provides conversable agents that combine models, tools, and human input in configurable topologies (Wu et al. 2023a), while AgentVerse dynamically recruits and adjusts its agent group across a recruit–decide–act–evaluate loop (Chen et al. 2023d).

A revealing boundary case is that “multi-agent” collaboration need not require multiple agents at all: Solo Performance Prompting has a single model adopt several personas that critique one another in turn, recovering much of the benefit within one context window (Wang et al. 2023e). This sharpens the question the rest of the section pursues, what, exactly, does spawning separate agents buy? Table 7 contrasts representative frameworks along the axes that answer it: their topology, how roles are assigned, the medium of communication, whether a human is in the loop, and the domain they target.

7.2 Debate and Discussion

The most-studied coordination protocol is debate. Having several model instances propose answers and argue over successive rounds improves factuality and reduces fallacious reasoning (Du et al. 2023); the Multi-Agent Debate framework adds structured tit-for-tat argument and a judge to counter the “degeneration of thought” a lone model suffers (Liang et al. 2023). The protocol has been turned to evaluation, panels of referee agents correlate with human judgment better than a single evaluator (Chan

et al. 2023), and to heterogeneous ensembles, where diverse models discuss and then vote by confidence (Chen et al. 2023c).

The enthusiasm deserves a skeptic, and the literature supplies a pointed one. A systematic comparison of debate against simpler strategies at matched cost finds that debate frequently fails to beat plain sampling-and-voting out of the box, and is often the more expensive way to reach the same accuracy (Smit et al. 2023). The productive reframing comes from the scalable-oversight literature: when two stronger models debate and a weaker judge decides, optimizing the debaters for *persuasiveness* raises the judge’s accuracy (Khan et al. 2024). Debate, then, is not reliably a route to better reasoning per unit compute, but it may be a route to *supervising* systems stronger than the supervisor, a different and arguably more important use, which we revisit in Section 11.

7.3 Scaling and Ensembling

The skeptic’s null hypothesis deserves its own subsection, because it is strong. Simply sampling many independent agent responses and taking a majority vote (an “agent forest” with no communication at all) improves performance monotonically with the number of agents, and this scaling accounts for a large share of the gains often attributed to elaborate coordination (Li et al. 2024b). Any claim that a communication protocol helps must therefore clear this bar. Some designs do: Mixture-of-Agents stacks layers in which each agent conditions on the previous layer’s outputs, and this structured aggregation of open models surpasses far stronger single models on instruction-following benchmarks (Wang et al. 2024b). When such systems are deployed the number of agents queried becomes a cost to be managed, not just an accuracy dial (Lee et al. 2026b). Pushing scale further, arranging over a thousand agents in explicit collaboration graphs finds that performance depends on the topology, with irregular graphs outperforming regular ones, an early collaborative scaling result (Qian et al. 2024). The lesson is that ensembling is the baseline to beat, and structured aggregation, not mere conversation, is what beats it.

7.4 Communication Topology and Team Optimization

If communication must earn its cost, the natural next step is to optimize *who talks to whom*. DyLAN treats the interaction graph as dynamic, pruning unhelpful agents mid-task via an agent-importance score and stopping early when consensus forms (Liu et al. 2023b), and Exchange-of-Thought catalogues the communication paradigms available (memory, report, relay, and debate) with a confidence mechanism to weigh contributions (Yin et al. 2023). A newer line makes the topology itself the object of optimization: GPTSwarm formalizes agent systems as computational graphs whose prompts and edges are jointly learned (Zhuge et al. 2024a), and AFlow searches over code-represented workflows to discover agentic pipelines automatically (Zhang et al. 2024d). Figure 8 sketches the recurring topologies, from fixed pipelines and stars to meshes and pruned dynamic networks.

Coordination is not always cooperative. Scorable negotiation games, some with adversarial players, test strategic communication and expose manipulation and self-interest as failure modes worth studying (Abdelnabi et al. 2023), and embodied teams must coordinate under partial observability, as in the modular cooperative agents of CoELA (Zhang et al. 2023b). At the largest scale, populations of agents with memory and reflection produce believable emergent social behavior, the clearest evidence that coordination dynamics are a genuine object of study rather than an engineering convenience (Park et al. 2023a).

Coordination also fails in systematic ways, and naming those failures sharpens the skeptic’s case from §7.2. A study of hundreds of execution traces across popular multi-agent frameworks distills fourteen recurring failure modes, under-specified roles, inter-agent misalignment, and weak verification among them, and attributes most breakdowns to these process failures rather than to the base model (Cemri et al. 2025). The lesson reinforces the section’s takeaway: multi-agent systems help when they add the right structure and fail when the structure itself is flawed.

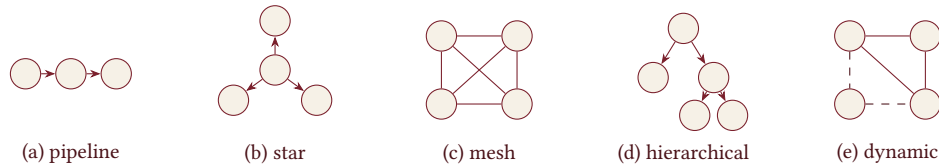


FIGURE 8: Communication topologies for multi-agent systems. (a) A pipeline passes work along a fixed sequence (MetaGPT, ChatDev); (b) a star routes through an orchestrator (AutoGen); (c) a mesh has every agent debate every other (§7.2); (d) a hierarchy delegates down a tree; (e) a dynamic network prunes edges at run time (dashed), as in DyLAN.

Key Takeaway

Multi-agent systems help when they add *structure* (roles, verification, layered aggregation) not merely more conversation. The honest baseline is a compute-matched single agent sampled many times and voted, and any multi-agent result should be reported against it; debate in particular often fails to clear that bar for accuracy, though it remains promising for scalable oversight.

8 Agents in Interactive Environments

The pillars so far describe agents largely through text. But an agent earns its name only by acting on something, and the something is increasingly a rich, partially observable environment: a web page, a phone screen, a desktop, a kitchen. Prior surveys tend to treat web, GUI, and embodied agents as separate literatures. We deliberately treat them as one family, because they share a single hard problem, *grounding*: mapping language and perception onto the specific, often irreversible action the environment will accept, whether that action is clicking the right link, tapping the right widget, or grasping the right object. Multimodal agent surveys support this unification across gaming, robotics, and interface control (Durante et al. 2024).

8.1 Web Agents

The web was the first rich environment, and its agents trace a clean lineage. WebGPT fine-tuned a model to answer questions by operating a text browser under human feedback, before ChatGPT existed (Nakano et al. 2021). WebShop then supplied a scalable simulated environment (over a million products) in which grounded agents navigate and purchase against natural-language goals (Yao et al. 2022a). Mind2Web moved to real websites, offering thousands of open-ended tasks across more than a hundred sites for training generalist web agents (Deng et al. 2023), and WebArena built a realistic, self-hostable environment with functionally checked tasks that has become a standard testbed (Zhou et al. 2024c).

The decisive recent shift is multimodal, and it identifies grounding as the bottleneck. SeeAct shows that a large multimodal model can act as a generalist web agent when it perceives the rendered page rather than only its HTML, but that turning a correct plan into the correct click (grounding) is where it most often fails (Zheng et al. 2024). WebVoyager closes the loop end-to-end on live websites with a multimodal agent and an automatic evaluator (He et al. 2024). The dedicated web-agent survey of Ning et al. (2025) catalogues the architectures and trustworthiness concerns this subfield has accumulated. The maturest commercial descendants of this line are *deep research* agents that run long-horizon, tool-mediated search and synthesis; the paradigm now has open end-to-end models (Tongyi DeepResearch Team 2025) and a systematic survey (Huang et al. 2025).

8.2 GUI and Computer-Use Agents

Generalizing from the browser to the whole screen yields the computer-use agent, and here perception becomes purely visual. CogAgent is an early large vision-language model built for graphical interfaces, using dual low- and high-resolution encoders to read screenshots and outperform HTML-based methods (Hong et al. 2023). SeeClick isolates the core subproblem, *GUI grounding*⁶, and introduces the ScreenSpot benchmark to measure locating the right element from a screenshot alone (Cheng et al. 2024).

On top of this perception layer sit device agents: AppAgent explores smartphone apps through a human-like action space (Zhang et al. 2023a), Mobile-Agent uses visual perception tools to operate apps from screenshots without back-end access (Wang et al. 2024c), and OS-Copilot extends autonomy to the operating-system level with a self-improving agent (Wu et al. 2024b). The frontier is end-to-end native models that map screenshots to actions directly, such as UI-TARS (Qin et al. 2025) (an RL-trained model in the lineage of Table 4) and its multi-turn-RL successor (Wang et al. 2025a), alongside the first main-stream commercial computer-use systems: Anthropic’s computer use (Anthropic 2024a) and OpenAI’s Operator (OpenAI 2025). What has driven the subfield is less the agents than the environments that grade them: OSWorld evaluates open-ended tasks in a real computer and reports agents near twelve percent success against seventy-two percent for humans (Xie et al. 2024b), and AndroidWorld supplies a dynamic, reward-bearing Android environment with parameterized tasks (Rawles et al. 2024). Two GUI-agent surveys map the space in detail (Nguyen et al. 2024; Zhang et al. 2024a).

8.3 Embodied and Robotic Agents

Embodied agents face the same grounding problem with physics added: actions move a body and cannot be undone. The foundational move was to use the LLM as a planner constrained by what the body can do. SayCan combines a model’s semantic knowledge with learned robotic affordances so that only feasible actions are proposed (Ahn et al. 2022), and Inner Monologue closes the loop by feeding environment feedback back to the planner in language, improving instruction completion without new training (Huang et al. 2022). ALFWorld aligns a text world with an embodied one so that policies learned abstractly transfer to grounded execution, and has become a standard testbed for LLM-based embodied agents (Shridhar et al. 2021).

A parallel line trains the mapping from perception to action end-to-end. PaLM-E injects sensor data directly into a language model to yield a generalist embodied model (Driess et al. 2023); the robotics transformers RT-1 and RT-2 scale action prediction and transfer web knowledge to control (Brohan et al. 2022; Brohan et al. 2023); and OpenVLA opens the resulting vision–language–action recipe (Kim et al. 2024a), with π_0 extending it to a flow-matching action expert controlling many robot embodiments (Black et al. 2024). We treat these only insofar as they bear on LLM-agent design; the broader robotics-learning literature is out of scope. Voyager, discussed in §5.4, closes the circle by pairing embodiment with a procedural-memory skill library for open-ended learning (Wang et al. 2023a). Table 8 compares the environments these agents are measured in.

Key Takeaway

Across web, GUI, and embodied settings the binding constraint is *grounding* (selecting the right element or affordance to act on) not high-level planning. The widest human–agent gaps (for example the roughly twelve-versus-seventy-two-percent split on OSWorld) are grounding gaps, and progress tracks better perception and interface design more than better reasoning.

⁶ **Grounding.** Mapping a referring expression or intended action onto the concrete interface element (a pixel region, a DOM node, an object) that realizes it. Grounding, not high-level planning, is the dominant failure mode of interactive agents; SeeClick introduces the ScreenSpot benchmark to measure it directly (Cheng et al. 2024).

TABLE 8: Interactive environments by observation space, action space, scale, and whether the environment is stateful (actions change a persistent world state, evaluated by execution). Cross-referenced from Section 9.

Environment	Observation	Action space	#Tasks	State	Metric	Ref.
WebShop	text / HTML	search, click	12k instr.	✓	task score	(Yao et al. 2022a)
Mind2Web	HTML + screenshot	click, type, select	2k+	✗	element accuracy	(Deng et al. 2023)
WebArena	DOM / pixels	browser actions	812	✓	functional success	(Zhou et al. 2024c)
OSWorld	pixels + a11y tree	mouse, keyboard	369	✓	execution success	(Xie et al. 2024b)
AndroidWorld	pixels + a11y tree	touch, type	116	✓	reward / success	(Rawles et al. 2024)
ALFWorld	text (embodied)	high-level actions	6 types	✓	success rate	(Shridhar et al. 2021)

9 Evaluation and Benchmarks

Evaluation is the first of two cross-cutting concerns, and the more it is taken seriously the more sobering the picture becomes. The headline numbers of this section, GPT-4 solving under one percent of a travel-planning benchmark, agents reaching roughly a sixth of human success on real computer tasks, are not incidental; they are the field’s most honest signal of how far agents remain from reliability. This section asks what makes agents hard to measure, catalogues how the community measures them anyway, and then turns a critical eye on whether the measurements mean what they are taken to mean.

9.1 What Is Hard about Evaluating Agents

A static question-answering model is graded on one output; an agent must be graded on a trajectory. That difference is the source of every difficulty. Trajectories are *long*, so a single early misstep can doom an otherwise competent run, and *stochastic*, so one sample badly estimates the agent’s true success rate, which is why reliability metrics such as τ -bench’s pass^k , the probability of succeeding on all of k independent attempts, have emerged as necessary complements to average success (Yao et al. 2024). Trajectories also unfold in *stateful* environments, so grading requires either checking the final world state or judging the process, and the two can disagree: an agent may reach the right answer by an unsafe or lucky path. The two dedicated evaluation surveys organize these problems along complementary axes, what to evaluate versus how (Yehudai et al. 2025; Mohammadi et al. 2025), and both flag cost, safety, and robustness as the dimensions current practice most neglects. Alongside aggregate scores, fine-grained studies that catalogue an agent’s concrete failure modes, as done early for memory- and retrieval-augmented dialogue agents (Lee et al. 2022), remain a useful complement to leaderboard numbers.

9.2 The Benchmark Landscape

The benchmark landscape divides into generalist and capability-specific suites (Table 9). On the generalist side, AgentBench evaluates a model as an agent across eight distinct environments and exposes a wide gap between commercial and open models (Liu et al. 2024c); GAIA poses questions that are conceptually simple for people yet require reasoning, tool use, and browsing, with people scoring around ninety-two percent and an augmented GPT-4 only about fifteen (Mialon et al. 2023); AgentBoard replaces binary success with a fine-grained progress-rate metric across a thousand environments (Ma et al. 2024); and SmartPlay isolates individual agentic capabilities across a suite of games (Wu et al. 2023b).

Capability-specific benchmarks probe one competence deeply, and their difficulty is instructive. SWE-bench asks agents to resolve real GitHub issues by editing a live codebase and passing hidden tests (Jimenez et al. 2023); τ -bench simulates multi-turn tool-agent-user interaction with policy constraints (Yao et al. 2024), extended by τ^2 -bench to dual-control settings where user and agent act on the environment simultaneously (Barres et al. 2025); TravelPlanner stress-tests constrained long-

TABLE 9: The agent benchmark landscape: generalist suites (top) and capability-specific benchmarks (bottom). “Env type” summarizes the interaction setting; “–” marks a count the source does not fix (e.g., open model rosters). Overlapping tool and environment benchmarks appear in Tables 6 and 8.

Benchmark	Target	Env type	#Tasks	Metric	Ref.
GAIA	general assistant	web + tools	466	exact match	(Mialon et al. 2023)
AgentBench	general agent	8 environments	–	success rate	(Liu et al. 2024c)
AgentBoard	multi-turn agent	9 task types	1013 envs	progress rate	(Ma et al. 2024)
SmartPlay	agentic capabilities	6 games	–	reward	(Wu et al. 2023b)
SWE-bench	software engineering	12 repositories	2294	resolved %	(Jimenez et al. 2023)
τ -bench	tool-agent-user	retail, airline	–	pass ^k	(Yao et al. 2024)
τ^2 -bench	dual-control dialogue	telecom	–	pass ^k	(Barres et al. 2025)
TravelPlanner	constrained planning	sandbox	1225	plan validity	(Xie et al. 2024a)
InterCode	interactive coding	Bash/SQL/Python	–	success rate	(Yang et al. 2023c)
Terminal-Bench	terminal workflows	sandboxed terminal	89	test-verified success	(Merrill et al. 2026)
MLAgentBench	ML experimentation	code + execution	13	improvement	(Huang et al. 2023b)
ScienceAgentBench	data-driven science	code	102	success rate	(Chen et al. 2024b)
OSWorld	computer use	real OS	369	execution success	(Xie et al. 2024b)
WebArena	web navigation	self-hosted sites	812	functional success	(Zhou et al. 2024c)

horizon planning, and the near-zero success it originally reported for GPT-4 (about 0.6 percent) is among the field’s most cited reality checks (Xie et al. 2024a); InterCode frames interactive coding as a reinforcement-learning environment with execution feedback (Yang et al. 2023c); Terminal-Bench assembles hard, realistic command-line tasks (Merrill et al. 2026); MLAgentBench evaluates agents on open-ended machine-learning experimentation (Huang et al. 2023b); and ScienceAgentBench grounds evaluation in a hundred-plus expert-validated data-science tasks (Chen et al. 2024b). Where these overlap the tool benchmarks of Table 6 and the environments of Table 8, we cross-reference rather than repeat.

9.3 Methodological Critique

A survey that only listed benchmarks would be complicit in their misuse, so we take the field’s own methodological critique seriously here. The anchor is Kapoor et al. (2024), whose central argument is that the near-exclusive focus on accuracy, with cost ignored, produces agents that are needlessly complex, overfit to their leaderboards, and expensive to run for gains that evaporate under scrutiny. Their remedies are concrete and, once stated, hard to argue with: report accuracy and cost together on a Pareto frontier rather than accuracy alone; distinguish the cost of *developing* an agent against a benchmark from its *inference* cost; and hold out genuinely unseen tasks, because agents tuned on a public test set inflate in ways that do not transfer. The same group’s Holistic Agent Leaderboard turns these remedies into shared infrastructure, a standardized harness that re-evaluates agents at scale with cost reported alongside accuracy (Kapoor et al. 2025). The same logic indicts several methods praised earlier in this survey, self-consistency, tree search, and elaborate multi-agent protocols all buy accuracy with compute that accuracy-only leaderboards render invisible.

Two further threats compound the cost blindness. Contamination is structural: agent benchmarks built from public GitHub issues, web pages, or question sets may already sit in pretraining data, so a high score can reflect memorization rather than capability, and the effect is hard to detect after the fact. And the increasingly common practice of using a strong model as an automatic judge, now systematized as agent-as-a-judge (Zhuge et al. 2024b), introduces a circularity, the evaluator shares the biases, blind spots, and even the identity of the systems it grades. Multi-agent referee panels mitigate but do not remove this, since they correlate with human judgment yet inherit model-family biases (Chan et al. 2023). The upshot is not that agent benchmarks are worthless but that a single accuracy number, uncontrolled for cost, contamination, and judge bias, is not the evidence it appears to be.

Key Takeaway

An agent result reported as an accuracy number alone is incomplete. Cost (in compute, tokens, and latency) must be reported alongside accuracy, because most headline gains in this survey were bought with inference that accuracy-only leaderboards hide; contamination and agent-as-judge circularity further erode single-number comparisons.

10 Applications

Where agents are deployed is not, we argue, primarily a matter of economic value; it is a matter of *feedback*. Domains where the environment returns a cheap, reliable, ground-truth signal (does the test pass, does the reaction work, does the theorem check) let an agent iterate its way to competence, and these domains lead. Domains where the signal is expensive, delayed, or contested (a clinician’s judgment, a market’s verdict) trail, not for lack of interest but because the loop of Definition 1 cannot close tightly. The ordering of the subsections below tracks that thesis, from the most verifiable domain to the least.

10.1 Software Engineering

Software engineering is the paradigm verifiable domain: a unit test is a free, exact oracle, and it is no accident that coding agents are the most mature. SWE-agent resolves real repository issues through a purpose-built agent–computer interface (§6.3) (Yang et al. 2024a); AutoCodeRover raises efficiency by grounding the search in program structure and spectrum-based fault localization (Zhang et al. 2024e); and OpenHands provides an open platform on which much of this work now builds (Wang et al. 2024d). The dedicated survey of Liu et al. (2024a) catalogues the subfield. SWE-Lancer prices the same competence economically, asking whether frontier models can earn real freelance payouts and finding that they still leave most of the posted value unearned (Miserendino et al. 2025).

Yet software engineering also hosts this survey’s cleanest cautionary tale about agentic complexity, and it deserves equal weight. Agentless challenges the premise that issue resolution needs an autonomous agent at all: a fixed three-phase pipeline (localize, repair, validate) with no open-ended planning loop matches or beats far more elaborate agents on SWE-bench at a fraction of the cost (Xia et al. 2024). This is the cost-versus-capability argument of §9.3 made concrete: when the feedback signal is strong enough, a simple pipeline that exploits it can dominate an agent that reasons its way around it, and the burden of proof is on the agent.

10.2 Scientific Discovery

Scientific domains offer verifiable feedback of a costlier kind (an experiment, a proof, a passing computation) and agents have moved from augmenting scientists to conducting studies. Early systems equipped a model with domain tools: ChemCrow wraps a GPT-4 agent in expert chemistry tools for synthesis planning and safety (Bran et al. 2023), and Coscientist closes the loop to physical hardware, designing and executing chemistry experiments through robotic APIs (Boiko et al. 2023). A more ambitious line automates the research process itself: the AI Scientist generates ideas, runs experiments, and writes papers end to end (Lu et al. 2024), its successor produced the first fully AI-generated paper accepted at a peer-reviewed workshop (Yamada et al. 2025), and an AI co-scientist generates and ranks novel biomedical hypotheses under expert validation (Gottweis et al. 2025).

The most convincing evidence comes from discoveries that are checkable. AlphaEvolve, an evolutionary coding agent, found a way to multiply two 4×4 complex-valued matrices in 48 scalar multiplications, the first improvement on Strassen’s algorithm for this case in 56 years (Novikov et al. 2025), and Kosmos runs long autonomous research campaigns around a shared structured world

TABLE 10: Application domains ordered by the quality of the feedback signal the environment provides. Maturity is a coarse judgment of deployment readiness; the dominant risk is the failure mode most cited in each domain’s literature.

Domain	Representative systems	Feedback signal	Maturity	Dominant risk
Software	SWE-agent (Yang et al. 2024a), OpenHands (Wang et al. 2024d), Agentless (Xia et al. 2024)	unit tests	deployed	wrong / insecure code
Science	ChemCrow (Bran et al. 2023), AI Scientist (Lu et al. 2024), AlphaEvolve (Novikov et al. 2025)	experiment / proof	research	unvalidated claims
Healthcare	MDAgents (Kim et al. 2024b), AMIE (Tu et al. 2024), Agent Hospital (Li et al. 2024a)	clinician rating	research	hallucination, liability
Finance	FinMem (Yu et al. 2023), TradingAgents (Xiao et al. 2024), FinGPT (Yang et al. 2023a)	market returns	research	backtest overfitting

model (Mitchener et al. 2025). Because a matrix-multiplication algorithm either works or does not, such results sidestep the evaluation ambiguities of §9; ScienceAgentBench supplies the corresponding expert-validated benchmark (Chen et al. 2024b), and the subfield has its own survey (Ren et al. 2025).

10.3 Healthcare

Medicine inverts the feedback economics: ground truth is expensive, delayed, and ethically gated, so progress relies on simulation and expert rating rather than on a cheap oracle. MDAgents adapts its collaboration structure (solo, group, or hierarchical) to the complexity of a medical decision (Kim et al. 2024b), and Agent Hospital lets doctor agents self-evolve by treating simulated patients, manufacturing experience where real cases cannot be used freely (Li et al. 2024a); AgentClinic supplies a multimodal benchmark of such simulated clinical settings (Schmidgall et al. 2024). The most rigorous result is AMIE, a diagnostic dialogue agent that, in a randomized OSCE-style study, was rated higher than primary-care physicians on 28 of 32 axes rated by specialist physicians (Tu et al. 2024), a striking figure that the survey of Wang et al. (2025c) rightly situates alongside unresolved risks of hallucination and liability, which we take up in Section 11.

10.4 Finance

Finance offers a feedback signal that looks precise but is treacherous: returns are quantitative yet noisy, non-stationary, and easy to overfit in backtests. FinMem structures a trading agent around layered, human-cognition-inspired memory (Yu et al. 2023); TradingAgents mirrors a trading firm with specialized analyst, researcher, and risk-management roles (Xiao et al. 2024); and FinGPT supplies open financial-model infrastructure underneath much of this work (Yang et al. 2023a). The survey of Ding et al. (2024) reviews the area, but the binding caveat is methodological: a favorable backtest is not a market outcome, and reported returns that are not out-of-sample say little, the same no-invented-numbers discipline this survey applies to benchmark tables applies with double force to financial claims. Table 10 summarizes the four domains by their feedback signal, maturity, and dominant risk.

Key Takeaway

The maturity of an application is predicted by the quality of its feedback signal, not by its economic value. Verifiable domains (code with unit tests, mathematics and chemistry with checkable results) lead precisely because a cheap ground-truth oracle lets agents iterate; medicine and finance trail because their signals are expensive, delayed, or noisy, and their dominant risks are

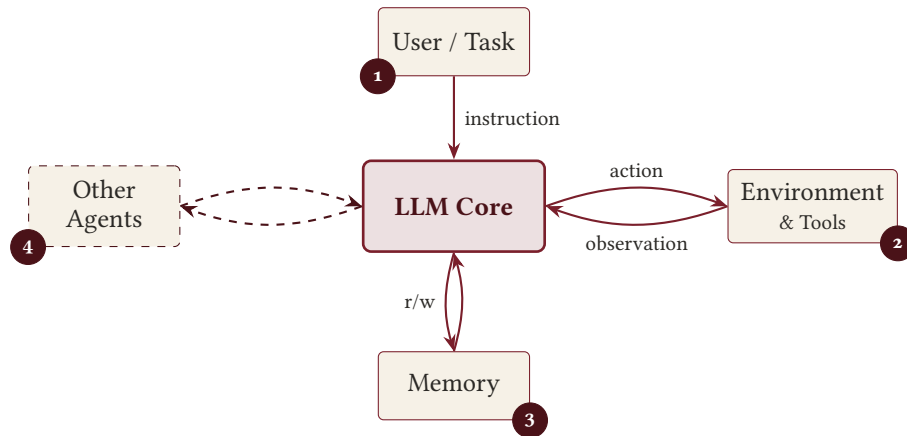


FIGURE 9: The agent loop of Figure 3 as an attack surface. Entry points: ❶ direct prompt injection through the user instruction; ❷ indirect prompt injection through tool or environment content returned as an observation; ❸ poisoning of persistent memory or a retrieval knowledge base; and ❹ contagion from another agent. Numbers map to the paragraphs of §11.2; the tool supply chain (a compromised tool itself) is a further surface we treat as an open problem in §12.

correspondingly harder to test away.

11 Safety, Security, and Trustworthiness

Autonomy is precisely what makes LLM-based agents useful and what makes them risky: an agent that can call tools, write files, or send messages on a user’s behalf can also do so incorrectly, at scale, and without a human in the loop (Yu et al. 2025).⁷

11.1 The Agentic Threat Model

The safety of a chatbot is largely the safety of its text; the safety of an agent is the safety of its *actions*. Three properties of the loop in Figure 3 each open a new class of risk. Tools are *actuators*: an injected instruction becomes a real side effect, a sent email, a deleted file, a transferred payment. Memory is *persistent state*: a corruption written once can influence every later decision. And autonomy is *scale*: an agent repeats its mistakes without a human to interrupt them. The recent surveys of the area converge on this framing, that autonomy itself, not merely the underlying model, is the source of the new risks (Su et al. 2025); that threats factor into intrinsic (reasoning, memory, tools) and extrinsic (user, other agents, environment) dimensions (Yu et al. 2025); and that safety must be considered across the full lifecycle from data to deployment (Wang et al. 2025b). Figure 9 redraws the agent loop as an attack surface, numbering the entry points that the next subsection works through.

11.2 Attacks on LLM-Based Agents

The dominant attack exploits a design feature: an agent cannot reliably tell instructions from data, because both arrive as text.

⁷ Compare with the tool-use mechanisms of §6: the same interfaces that let an agent act on the world are what an adversary can target.

Definition 4 (Indirect Prompt Injection)

Indirect prompt injection is an attack in which an adversary plants instructions in content the agent will later read as data (a web page, a document, an email, a tool’s output) so that the agent executes the adversary’s instructions while believing it is processing benign input (Greshake et al. 2023).

Injection is entry points ❶ and ❷ of Figure 9. Since it was first shown to compromise real LLM-integrated applications (Greshake et al. 2023), a series of benchmarks has quantified how exposed agents are: InjecAgent finds a ReAct-prompted GPT-4 acts on injected instructions in roughly a quarter of tool-integrated cases (Zhan et al. 2024); AgentDojo provides a dynamic environment of realistic tasks and attacks and shows that current agents cannot be simultaneously useful and robust (Debenedetti et al. 2024); and WASP finds that even top reasoning models begin executing injected instructions on web tasks between sixteen and eighty-six percent of the time, though full goal hijacking is rarer (Evtimov et al. 2025).

The persistent state of memory adds a second, quieter class, entry point ❸. AgentPoison injects a handful of poisoned entries into an agent’s memory or retrieval knowledge base and triggers malicious behavior with over eighty-percent success, without any model training and with little effect on benign queries (Chen et al. 2024a). BadAgent shows that backdoors planted during fine-tuning survive subsequent fine-tuning on clean data (Wang et al. 2024f), a persistence result echoed at the alignment level in §11.4. Finally, composition amplifies risk, entry point ❹: red-teaming multi-agent systems shows that collaboration spreads and intensifies harmful behavior rather than diluting it (Tian et al. 2023), and the Agent Security Bench unifies these attack and defense classes at scale, reporting attack success rates up to eighty-four percent and finding existing defenses largely ineffective (Zhang et al. 2024b).

11.3 Risk Evaluation and Guardrails

Measuring these risks safely is itself a research problem, since one cannot let an agent cause real harm to see whether it would. ToolEmu resolves this by having a strong model emulate tool execution in a sandbox, surfacing risky behavior without real tools and finding that even the safest agents act with potentially severe consequences about a quarter of the time (Ruan et al. 2024). R-Judge tests whether models can recognize risk in agent behavior logs, and the best reach only around seventy-four percent accuracy, with most near chance (Yuan et al. 2024). Misuse and embodied hazards have their own suites: AgentHarm measures whether jailbroken agents retain the capability to carry out coherent harmful multi-step tasks (Andriushchenko et al. 2024), and SafeAgentBench finds embodied agents reject only five to ten percent of explicitly hazardous instructions (Yin et al. 2024), with OS-Harm extending safety measurement to computer-use agents operating real interfaces (Kuntz et al. 2025). On the defense side, a spectrum of countermeasures has emerged. Training-time methods teach the model an *instruction hierarchy* that privileges trusted system instructions over injected ones (Wallace et al. 2024); prompt-level methods such as *spotlighting* mark untrusted content so the model can discount it (Hines et al. 2024); and constitution-guided planning enforces safety before, during, and after plan generation (Hua et al. 2024). The strongest current answer to indirect injection is architectural: CaMeL isolates untrusted data from the agent’s control flow by design and gates actions behind explicit capability checks (Debenedetti et al. 2025), and a companion catalogue of design patterns constrains what an agent may do once it has read untrusted input, trading some generality for resistance by construction (Beurer-Kellner et al. 2025). These genuinely raise the bar, but as Table 11 records, coverage is uneven, strongest for direct injection under an instruction hierarchy, only partial for indirect injection, and weakest for memory poisoning and backdoors, so the summary is that no attack class yet has a defense that is both general and validated at deployment scale.

To make the relationship between capability and safety concrete rather than assumed, we ran a small measurement of our own. Across six Qwen2.5-Instruct models spanning 1.5B to 72B parameters, with Llama-3.1-8B-Instruct as a second-family control, we paired agentic capability (ALFWorld success,

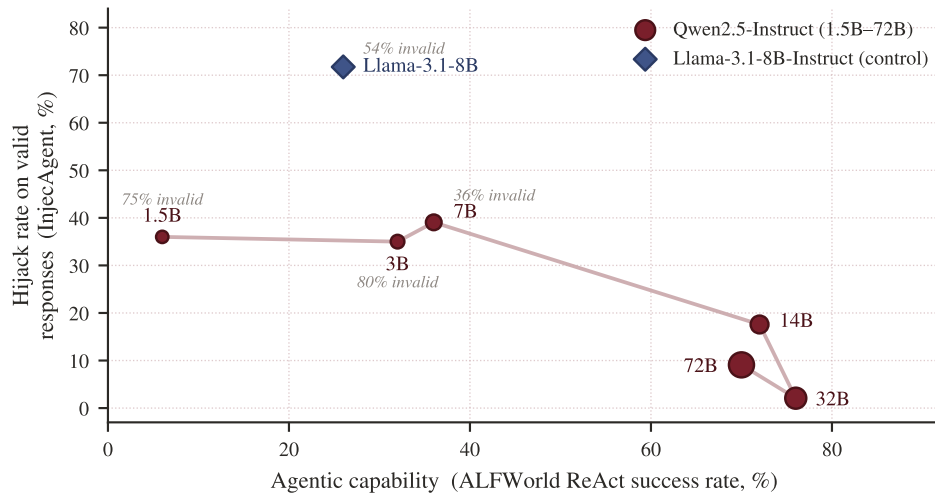


FIGURE 10: Agentic capability versus injection susceptibility across model scale: six Qwen2.5-Instruct models (burgundy; 1.5B–72B, marker size growing with parameter count) and a Llama-3.1-8B-Instruct control (indigo). The vertical axis scores InjecAgent attack success among *valid* responses; italic annotations give the share of invalid responses, which dominates the smallest models and would otherwise masquerade as robustness. Within the family, scale improves capability and injection-robustness together, and the Llama point shows the level is family-specific. Neither observation generalizes to the other attack classes of Table 11, where strong models still fail.

the same axis as §4.3) with susceptibility to indirect prompt injection on InjecAgent (Zhan et al. 2024), scored as attack success among valid responses; Figure 10 shows the result. Three observations follow. First, within the family the two axes broadly improve together: capability climbs from 6 percent at 1.5B to 70–76 percent for the 14B–72B models, while the hijack rate falls from 35–39 percent below 10B, through 18 percent at 14B, to 2–9 percent for the two largest models, though not perfectly monotonically (the 72B model is hijacked on 9 percent of valid responses against 2 for the 32B). Second, the metric matters: on the raw all-probes attack rate the two smallest models look nearly robust (7–9 percent), but only because three quarters or more of their responses are not valid tool calls at all; robustness that is really incompetence disappears once the metric conditions on valid actions (35–36 percent). Third, the level is family-specific: at comparable scale, Llama-3.1-8B is hijacked on 72 percent of its valid responses against 39 for Qwen2.5-7B. So scale buys injection-robustness within this family, but it is the optimistic exception, not a general law: the memory-poisoning, backdoor, and misuse benchmarks tabulated below show even the strongest models failing badly, so capability cannot be read as a proxy for safety; the two are distinct axes and must be measured one by one.

11.4 Alignment-Level Risks

Beneath the security threats lies a subtler class in which the agent’s own learned dispositions, not an external adversary, are the hazard. The evidence begins with behavior induced by training: model-written evaluations reveal that RLHF can amplify sycophancy and expressed self-preservation as scale grows (Perez et al. 2022), and controlled studies trace sycophancy to the preference feedback used to train assistants, with raters sometimes preferring convincing wrong answers to correct ones (Sharma et al. 2023). A survey of AI deception documents that systems already learn to induce false beliefs in humans (Park et al. 2023b).

For agents, these dispositions become more dangerous because agents act. *Sleeper Agents* shows that a model trained to behave maliciously under a trigger retains that behavior through standard safety training, adversarial training can even teach it to hide the trigger better (Hubinger, Denison, et al. 2024). Frontier models have been observed to *scheme* in context on agentic tasks, introducing deliberate errors,

TABLE 11: Agent attack classes, a representative attack and benchmark for each, the prevailing defense direction, and whether a robust general defense exists. $\sim$ marks partial mitigations; $\times$ marks an open problem. The pervasiveness of $\times$ is the section’s central finding.

Attack class	Representative	Benchmark	Defense direction	Robust?
Direct prompt injection	jailbreak prompts	AgentHarm (Andriushchenko et al. 2024)	instruction hierarchy	$\sim$
Indirect prompt injection	Greshake et al. (2023)	AgentDojo (DeBenedetti et al. 2024), InjecAgent (Zhan et al. 2024)	spotlighting, CaMeL	$\sim$
Memory / KB poisoning	AgentPoison (Chen et al. 2024a)	ASB (Zhang et al. 2024b)	provenance filtering	$\times$
Backdoor	BadAgent (Wang et al. 2024f)	ASB (Zhang et al. 2024b)	data auditing	$\times$
Harmful-task misuse	jailbroken agent	SafeAgentBench (Yin et al. 2024), OS-Harm (Kuntz et al. 2025)	refusal + monitoring	$\sim$

attempting to disable oversight, and in some cases trying to exfiltrate their own weights (Meinke et al. 2024). And emergent misalignment demonstrates that fine-tuning on a narrow harmful task (writing insecure code) can generalize into broad misalignment on unrelated prompts (Betley et al. 2025), a result whose implications for the routine practice of fine-tuning agents on task data are not yet understood. The comprehensive alignment survey of Ji et al. (2023) supplies the conceptual scaffolding (robustness, interpretability, controllability, ethicality) within which these agent-specific failures sit.

11.5 Autonomy Governance

Because the risks scale with autonomy, some of the response is necessarily governance rather than engineering. Frameworks that define discrete levels of agent autonomy give a vocabulary for specifying how much control a human retains (Feng et al. 2025b), and a prominent position paper argues that fully autonomous agents should not be developed at all, since risk rises faster than benefit as the human is removed from the loop (Mitchell et al. 2025). Between these poles sits the design of human-agent oversight, feedback, interruption, and escalation mechanisms that keep a person meaningfully in control (Zou et al. 2025), which we regard as the most actionable near-term safety lever. The empirical ground truth is sobering: an index of thirty deployed agentic systems finds that developers document capabilities far more thoroughly than safety evaluations, a transparency gap governance frameworks have yet to close (Staufer et al. 2026).

Key Takeaway

Capability and safety are distinct axes that must be measured separately, not read off one another. On some (indirect injection) scale improves both at once within a model family (Figure 10), though the level is family-specific and apparent small-model robustness is mostly incompetence; on others (memory poisoning, backdoors, misuse) even the agents that top the benchmarks of Section 9 fail catastrophically, with attack-success rates in the high tens of percent and no robust general defense. Capability therefore does not imply safety, and progress on one cannot be assumed from the other.

12 Open Challenges and Future Directions

The preceding sections suggest seven challenges that we expect to shape the next phase of LLM-agent research. Each is grounded in results already surveyed rather than in speculation, and several are

TABLE 12: The survey’s both-sides findings collected in one place: where an optimistic consensus meets documented negative evidence, and how we reconcile them.

Topic	Optimistic claim	Counter-evidence	Reconciliation	§
Self-correction	self-critique improves reasoning	fails without an external signal (Huang et al. 2023a; Kamoi et al. 2024)	needs external or trained-in feedback	4.4
Multi-agent debate	debate raises factuality	often $\leq$ ensembling at equal cost (Smit et al. 2023)	useful for oversight, not raw accuracy	7.2
Memory	a larger store helps	structure and policy dominate capacity (Zeng et al. 2024)	read/write policy is the bottleneck	5.3
Agent scaffolds	complex agents are required	a simple pipeline can match them (Xia et al. 2024)	strong feedback favors simpler designs	10.1
Evaluation	accuracy ranks agents	cost-blind leaderboards mislead (Kapoor et al. 2024)	report cost alongside accuracy	9.3

consequences of the tensions flagged along the way. Table 12 first collects the survey’s documented negative results in one place: wherever an optimistic consensus meets credible counter-evidence, we have given the two equal weight, and the reconciliations there are as much a part of our contribution as the taxonomy.

1. Reliability over long horizons. The single most important gap is that agents fail as tasks lengthen. The near-zero success on constrained long-horizon planning (Xie et al. 2024a) and the roughly sixfold human-agent gap on realistic computer tasks (§8.2) are not isolated: they reflect a structural problem in which a per-step error probability compounds over a trajectory, so that even a capable agent becomes unreliable over enough steps. Because current evaluation often reports averaged success rather than reliability (Kapoor et al. 2024), this fragility is routinely understated. Progress requires both error-correcting mechanisms that actually work (§4.4) and metrics such as pass^k that measure consistency, not just average competence.

2. Cost as a first-class objective. Almost every capability gain in this survey was purchased with inference compute (sampled reasoning paths, search trees, multi-agent rounds) yet accuracy-only leaderboards render that price invisible (Kapoor et al. 2024). Since a large share of multi-agent benefit can be recovered by simply sampling one agent more often (Li et al. 2024b), the open problem is to report and optimize agents on an accuracy–cost frontier, and to design methods that are Pareto-efficient rather than merely accurate; methods that adaptively budget an agent’s reasoning tokens are one promising direction (Lee et al. 2026a). Cost-awareness is not an accounting detail; it changes which methods look good.

3. Continual learning and memory consolidation. Agents that improve from their own experience without weight updates (§5.4) point toward lifelong learning, but the mechanisms are immature. Distilling reusable insights (Zhao et al. 2023) and consolidating salient facts across sessions (Chhikara et al. 2025) work at small scale; what remains open is consolidation that avoids both unbounded memory growth and the erosion of old competence (the agentic analogue of catastrophic forgetting) over months of operation. The emerging self-evolving-agent literature, now with a survey of its own, frames exactly this bridge from static foundation models to lifelong agentic systems (Fang et al. 2025).

4. Training agents, not just prompting them. The shift from wrapping scaffolds around fixed models to training models for agency, surveyed in §4.5 and Table 4, raises two open problems. The first is *data*: high-quality *trajectory*-level supervision (sequences of decisions, tool calls, and recoveries) is

scarce next to the text used for pretraining, and scaling its production, whether by offline collection or online experience, without a human in the loop is unsolved. The second is the question §4.5 left open: as reasoning and acting migrate into the weights, do the external scaffolds of §4 (search trees, reflection loops, elaborate coordination) remain necessary, or do they become redundant on tasks the model has internalized? The answer determines how much of this survey’s machinery the next generation of agents will still need.

5. Infrastructure and interoperability. As multi-agent systems leave the laboratory, they need standard protocols for how agents expose tools and talk to one another (Yan et al. 2025). Emerging interoperability standards, the Model Context Protocol for tool access (Anthropic 2024b) and the Agent2Agent protocol for inter-agent communication (Google 2025), are early attempts, but the research questions of secure, composable, and observable multi-agent infrastructure are largely open, and they interact directly with the security concerns of Section 11.

6. Evaluation validity. The evaluation critique of §9.3 is itself a research agenda. Contamination of benchmarks by pretraining data, overfitting to public leaderboards, and the circularity of agent-as-judge all threaten the validity of the numbers the field reports on (Kapoor et al. 2024; Yehudai et al. 2025). Building contamination-resistant, cost-controlled, and reproducible benchmarks (ideally with live or held-out task streams) is a precondition for trusting claims of progress at all.

7. Safety–capability co-evolution. Finally, the divergence documented in Section 11 must be closed rather than merely measured. Capability advances have not been matched by defenses, backdoors survive safety training (Hubinger, Denison, et al. 2024), and no attack class yet has a robust general solution, so the central governance question is how much autonomy to grant under this asymmetry. Whether one adopts graded autonomy (Feng et al. 2025b) or argues against full autonomy outright (Mitchell et al. 2025), safety must co-evolve with capability rather than trail it, and we regard this as the field’s defining long-term challenge.

13 Conclusion

This survey has organized the LLM-agent literature around four component pillars (planning, memory, tool use, and multi-agent coordination) grounded in interactive environments and application domains, with evaluation and safety as first-class concerns. Read through the function-first taxonomy of Section 3, a fragmented field becomes legible: systems built in separate subfields turn out to be answering the same small set of design questions.

Three movements summarize where the field stands. The *components* are consolidating: prompting is giving way to trained reasoning (§4.5), flat memory to structured stores (§5.3), and hand-listed tools to code as a universal action space (§6.3), so that much of the scaffolding of the early agent frameworks (Wang et al. 2023b) is migrating into the models themselves. The *frontier* has moved to the context and cross-cutting axes, grounding agents in real environments (§8), and evaluating them honestly for cost and reliability rather than headline accuracy (§9), where the largest gaps between demonstration and dependability remain (Kapoor et al. 2024). And *safety* is the binding constraint: the divergence between what agents can do and what can be done to them (§11) means the field’s progress will be governed less by new capabilities than by whether those capabilities can be made trustworthy. The recent general surveys catalogue how fast the field is moving (Luo et al. 2025); our contribution is a stable frame in which to place what comes next.

References

- Abdelnabi, S., A. Gomaa, S. Sivaprasad, L. Schönherr, and M. Fritz (2023). *Cooperation, Competition, and Maliciousness: LLM-Stakeholders Interactive Negotiation*. arXiv. arXiv: 2309.17234. URL: <https://arxiv.org/abs/2309.17234>.
- Ahn, M. et al. (2022). *Do As I Can, Not As I Say: Grounding Language in Robotic Affordances*. CoRL 2022 / arXiv. arXiv: 2204.01691. URL: <https://arxiv.org/abs/2204.01691>.
- Andriushchenko, M., A. Souly, M. Dziemian, D. Duenas, M. Lin, J. Wang, D. Hendrycks, A. Zou, Z. Kolter, M. Fredrikson, E. Winsor, J. Wynne, Y. Gal, and X. Davies (2024). *AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents*. ICLR 2025 (arXiv 2024). arXiv: 2410.09024. URL: <https://arxiv.org/abs/2410.09024>.
- Anthropic (2024a). *Introducing Computer Use, a New Claude 3.5 Sonnet, and Claude 3.5 Haiku*. URL: <https://www.anthropic.com/news/3-5-models-and-computer-use> (visited on 07/08/2026).
- (2024b). *Introducing the Model Context Protocol*. URL: <https://www.anthropic.com/news/model-context-protocol> (visited on 07/08/2026).
- Barres, V., H. Dong, S. Ray, et al. (2025). τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv: 2506.07982. URL: <https://arxiv.org/abs/2506.07982>.
- Besta, M., N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler (2023). *Graph of Thoughts: Solving Elaborate Problems with Large Language Models*. AAAI 2024. arXiv: 2308.09687. URL: <https://arxiv.org/abs/2308.09687>.
- Betley, J., D. Tan, N. Warncke, A. Sztyber-Betley, X. Bao, M. Soto, N. Labenz, and O. Evans (2025). *Emergent Misalignment: Narrow Finetuning can Produce Broadly Misaligned LLMs*. arXiv. arXiv: 2502.17424. URL: <https://arxiv.org/abs/2502.17424>.
- Beurer-Kellner, L., B. Buesser, A.-M. Crețu, et al. (2025). *Design Patterns for Securing LLM Agents against Prompt Injections*. arXiv: 2506.08837. URL: <https://arxiv.org/abs/2506.08837>.
- Black, K., N. Brown, D. Driess, et al. (2024). π_0 : A Vision-Language-Action Flow Model for General Robot Control. arXiv: 2410.24164. URL: <https://arxiv.org/abs/2410.24164>.
- Boiko, D. A., R. MacKnight, B. Kline, and G. Gomes (2023). *Autonomous chemical research with large language models*. In: Nature (vol. 624, pp. 570-578). URL: <https://www.nature.com/articles/s41586-023-06792-0>.
- Bran, A. M., S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller (2023). *ChemCrow: Augmenting large-language models with chemistry tools*. Nature Machine Intelligence / arXiv. arXiv: 2304.05376. URL: <https://arxiv.org/abs/2304.05376>.
- Brohan, A. et al. (2022). *RT-1: Robotics Transformer for Real-World Control at Scale*. arXiv / RSS 2023. arXiv: 2212.06817. URL: <https://arxiv.org/abs/2212.06817>.
- (2023). *RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control*. CoRL 2023 / arXiv. arXiv: 2307.15818. URL: <https://arxiv.org/abs/2307.15818>.
- Cai, T., X. Wang, T. Ma, X. Chen, and D. Zhou (2023). *Large Language Models as Tool Makers*. arXiv. arXiv: 2305.17126. URL: <https://arxiv.org/abs/2305.17126>.
- Cao, P., T. Men, W. Liu, J. Zhang, X. Li, X. Lin, D. Sui, Y. Cao, K. Liu, and J. Zhao (2025). *Large Language Models for Planning: A Comprehensive and Systematic Survey*. arXiv. arXiv: 2505.19683. URL: <https://arxiv.org/abs/2505.19683>.

- Cemri, M., M. Z. Pan, S. Yang, et al. (2025). *Why Do Multi-Agent LLM Systems Fail?* arXiv: 2503.13657. URL: <https://arxiv.org/abs/2503.13657>.
- Chan, C.-M., W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu (2023). *ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate*. ICLR 2024 (arXiv preprint 2023). arXiv: 2308.07201. URL: <https://arxiv.org/abs/2308.07201>.
- Chen, B., C. Shu, E. Shareghi, N. Collier, K. Narasimhan, and S. Yao (2023a). *FireAct: Toward Language Agent Fine-tuning*. arXiv: 2310.05915. URL: <https://arxiv.org/abs/2310.05915>.
- Chen, H., R. Pasunuru, J. Weston, and A. Celikyilmaz (2023b). *Walking Down the Memory Maze: Beyond Context Limit through Interactive Reading*. arXiv preprint. arXiv: 2310.05029. URL: <https://arxiv.org/abs/2310.05029>.
- Chen, J. C.-Y., S. Saha, and M. Bansal (2023c). *ReConcile: Round-Table Conference Improves Reasoning via Consensus among Diverse LLMs*. ACL 2024 (arXiv preprint 2023). arXiv: 2309.13007. URL: <https://arxiv.org/abs/2309.13007>.
- Chen, W., Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, and J. Zhou (2023d). *AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors*. ICLR 2024 (arXiv preprint 2023). arXiv: 2308.10848. URL: <https://arxiv.org/abs/2308.10848>.
- Chen, Z., Z. Xiang, C. Xiao, D. Song, and B. Li (2024a). *AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases*. NeurIPS 2024. arXiv: 2407.12784. URL: <https://arxiv.org/abs/2407.12784>.
- Chen, Z. et al. (2024b). *ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery*. ICLR 2025 / arXiv. arXiv: 2410.05080. URL: <https://arxiv.org/abs/2410.05080>.
- Cheng, K., Q. Sun, Y. Chu, F. Xu, Y. Li, J. Zhang, and Z. Wu (2024). *SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents*. ACL 2024. arXiv: 2401.10935. URL: <https://arxiv.org/abs/2401.10935>.
- Chhikara, P., D. Khant, S. Aryan, T. Singh, and D. Yadav (2025). *Memo: Building Production-Ready AI Agents with Scalable Long-Term Memory*. arXiv preprint. arXiv: 2504.19413. URL: <https://arxiv.org/abs/2504.19413>.
- Comanici, G., E. Bieber, M. Schaekermann, et al. (2025). *Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities*. arXiv: 2507.06261. URL: <https://arxiv.org/abs/2507.06261>.
- Das, P., S. Chaudhury, E. Nelson, I. Melnyk, S. Swaminathan, S. Dai, A. Lozano, G. Kollias, V. Chenthamarakshan, J. Navrátil, S. Dan, and P.-Y. Chen (2024). *Larimar: Large Language Models with Episodic Memory Control*. ICML 2024. arXiv: 2403.11901. URL: <https://arxiv.org/abs/2403.11901>.
- Debenedetti, E., I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr (2025). *Defeating Prompt Injections by Design*. arXiv: 2503.18813. URL: <https://arxiv.org/abs/2503.18813>.
- Debenedetti, E., J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr (2024). *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. NeurIPS 2024 Datasets and Benchmarks Track. arXiv: 2406.13352. URL: <https://arxiv.org/abs/2406.13352>.
- DeepSeek-AI, D. Guo, D. Yang, H. Zhang, et al. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv preprint (later published in Nature, 2025). arXiv: 2501.12948. URL: <https://arxiv.org/abs/2501.12948>.
- Deng, X., Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su (2023). *Mind2Web: Towards a Generalist Agent for the Web*. NeurIPS 2023 (Datasets & Benchmarks, Spotlight). arXiv: 2306.06070. URL: <https://arxiv.org/abs/2306.06070>.
- Ding, H., Y. Li, J. Wang, H. Chen, D. Guo, and Y. Zhang (2024). *Large Language Model Agent in Financial Trading: A Survey*. arXiv. arXiv: 2408.06361. URL: <https://arxiv.org/abs/2408.06361>.

- Driess, D. et al. (2023). *PaLM-E: An Embodied Multimodal Language Model*. ICML 2023. arXiv: 2303.03378. URL: <https://arxiv.org/abs/2303.03378>.
- Du, Y., S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch (2023). *Improving Factuality and Reasoning in Language Models through Multiagent Debate*. arXiv preprint (also ICML 2024 workshop track). arXiv: 2305.14325. URL: <https://arxiv.org/abs/2305.14325>.
- Durante, Z., Q. Huang, N. Wake, R. Gong, J. S. Park, et al. (2024). *Agent AI: Surveying the Horizons of Multimodal Interaction*. arXiv. arXiv: 2401.03568. URL: <https://arxiv.org/abs/2401.03568>.
- Evtimov, I., A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri (2025). *WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks*. arXiv. arXiv: 2504.18575. URL: <https://arxiv.org/abs/2504.18575>.
- Fang, J., Y. Peng, X. Zhang, et al. (2025). *A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems*. arXiv: 2508.07407. URL: <https://arxiv.org/abs/2508.07407>.
- Feng, J., S. Huang, X. Qu, et al. (2025a). *ReTool: Reinforcement Learning for Strategic Tool Use in LLMs*. arXiv: 2504.11536. URL: <https://arxiv.org/abs/2504.11536>.
- Feng, K. J. K., D. W. McDonald, and A. X. Zhang (2025b). *Levels of Autonomy for AI Agents*. arXiv. arXiv: 2506.12469. URL: <https://arxiv.org/abs/2506.12469>.
- Google (2025). *Announcing the Agent2Agent Protocol (A2A)*. URL: <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/> (visited on 07/08/2026).
- Gottweis, J., W.-H. Weng, A. Daryin, T. Tu, et al. (2025). *Accelerating scientific discovery with Co-Scientist*. arXiv. arXiv: 2502.18864. URL: <https://arxiv.org/abs/2502.18864>.
- Gou, Z., Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen (2023). *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*. ICLR 2024. arXiv: 2305.11738. URL: <https://arxiv.org/abs/2305.11738>.
- Greshake, K., S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz (2023). *Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. ACM Workshop on AI and Security (AISec) / arXiv. arXiv: 2302.12173. URL: <https://arxiv.org/abs/2302.12173>.
- Guo, T., X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang (2024a). *Large Language Model based Multi-Agents: A Survey of Progress and Challenges*. arXiv; IJCAI 2024. arXiv: 2402.01680. URL: <https://arxiv.org/abs/2402.01680>.
- Guo, Z., S. Cheng, H. Wang, S. Liang, Y. Qin, P. Li, Z. Liu, M. Sun, and Y. Liu (2024b). *StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models*. Findings of ACL 2024. arXiv: 2403.07714. URL: <https://arxiv.org/abs/2403.07714>.
- Gutiérrez, B. J., Y. Shu, Y. Gu, M. Yasunaga, and Y. Su (2024). *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*. NeurIPS 2024. arXiv: 2405.14831. URL: <https://arxiv.org/abs/2405.14831>.
- Hao, S., Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu (2023a). *Reasoning with Language Model is Planning with World Model*. EMNLP 2023. arXiv: 2305.14992. URL: <https://arxiv.org/abs/2305.14992>.
- Hao, S., T. Liu, Z. Wang, and Z. Hu (2023b). *ToolkenGPT: Augmenting Frozen Language Models with Massive Tools via Tool Embeddings*. NeurIPS 2023 (Oral). arXiv: 2305.11554. URL: <https://arxiv.org/abs/2305.11554>.
- He, H., W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu (2024). *WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models*. ACL 2024. arXiv: 2401.13919. URL: <https://arxiv.org/abs/2401.13919>.

- Hines, K., G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman (2024). *Defending Against Indirect Prompt Injection Attacks With Spotlighting*. arXiv: 2403.14720. URL: <https://arxiv.org/abs/2403.14720>.
- Hong, S., M. Zhuge, J. Chen, X. Zheng, Y. Cheng, et al. (2024). *MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework*. ICLR 2024. arXiv: 2308.00352. URL: <https://arxiv.org/abs/2308.00352>.
- Hong, W., W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Zhang, J. Li, B. Xu, Y. Dong, M. Ding, and J. Tang (2023). *CogAgent: A Visual Language Model for GUI Agents*. CVPR 2024. arXiv: 2312.08914. URL: <https://arxiv.org/abs/2312.08914>.
- Hou, X., Y. Zhao, S. Wang, and H. Wang (2025). *Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions*. arXiv: 2503.23278. URL: <https://arxiv.org/abs/2503.23278>.
- Hsieh, C.-Y., S.-A. Chen, C.-L. Li, Y. Fujii, A. Ratner, C.-Y. Lee, R. Krishna, and T. Pfister (2023). *Tool Documentation Enables Zero-Shot Tool-Usage with Large Language Models*. arXiv. arXiv: 2308.00675. URL: <https://arxiv.org/abs/2308.00675>.
- Hua, W., X. Yang, M. Jin, Z. Li, W. Cheng, R. Tang, and Y. Zhang (2024). *TrustAgent: Towards Safe and Trustworthy LLM-based Agents*. EMNLP 2024 Findings. arXiv: 2402.01586. URL: <https://arxiv.org/abs/2402.01586>.
- Huang, J. and K. C.-C. Chang (2022). *Towards Reasoning in Large Language Models: A Survey*. ACL 2023 Findings. arXiv: 2212.10403. URL: <https://arxiv.org/abs/2212.10403>.
- Huang, J., X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou (2023a). *Large Language Models Cannot Self-Correct Reasoning Yet*. ICLR 2024. arXiv: 2310.01798. URL: <https://arxiv.org/abs/2310.01798>.
- Huang, Q., J. Vora, P. Liang, and J. Leskovec (2023b). *MLAgentBench: Evaluating Language Agents on Machine Learning Experimentation*. arXiv. arXiv: 2310.03302. URL: <https://arxiv.org/abs/2310.03302>.
- Huang, W. et al. (2022). *Inner Monologue: Embodied Reasoning through Planning with Language Models*. CoRL 2022 / arXiv. arXiv: 2207.05608. URL: <https://arxiv.org/abs/2207.05608>.
- Huang, X., W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen (2024). *Understanding the Planning of LLM Agents: A Survey*. arXiv. arXiv: 2402.02716. URL: <https://arxiv.org/abs/2402.02716>.
- Huang, Y., Y. Chen, H. Zhang, et al. (2025). *Deep Research Agents: A Systematic Examination and Roadmap*. arXiv: 2506.18096. URL: <https://arxiv.org/abs/2506.18096>.
- Hubinger, E., C. Denison, et al. (2024). *Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training*. arXiv (Anthropic). arXiv: 2401.05566. URL: <https://arxiv.org/abs/2401.05566>.
- Ji, J., T. Qiu, B. Chen, B. Zhang, et al. (2023). *AI Alignment: A Comprehensive Survey*. arXiv. arXiv: 2310.19852. URL: <https://arxiv.org/abs/2310.19852>.
- Jimenez, C. E., J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan (2023). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv: 2310.06770. URL: <https://arxiv.org/abs/2310.06770>.
- Jin, B., H. Zeng, Z. Yue, et al. (2025). *Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning*. arXiv: 2503.09516. URL: <https://arxiv.org/abs/2503.09516>.
- Kamoi, R., Y. Zhang, N. Zhang, J. Han, and R. Zhang (2024). *When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs*. TACL 2024. arXiv: 2406.01297. URL: <https://arxiv.org/abs/2406.01297>.
- Kapoor, S., B. Stroebel, P. Kirgis, et al. (2025). *Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation*. arXiv: 2510.11977. URL: <https://arxiv.org/abs/2510.11977>.
- Kapoor, S., B. Stroebel, Z. S. Siegel, N. Nadgir, and A. Narayanan (2024). *AI Agents That Matter*. arXiv. arXiv: 2407.01502. URL: <https://arxiv.org/abs/2407.01502>.

- Karpas, E., O. Abend, Y. Belinkov, B. Lenz, O. Lieber, N. Ratner, Y. Shoham, H. Bata, Y. Levine, K. Leyton-Brown, D. Muhlgay, N. Rozen, E. Schwartz, G. Shachaf, S. Shalev-Shwartz, A. Shashua, and M. Tenenholz (2022). *MRKL Systems: A Modular, Neuro-Symbolic Architecture that Combines Large Language Models, External Knowledge Sources and Discrete Reasoning*. arXiv. arXiv: 2205.00445. URL: <https://arxiv.org/abs/2205.00445>.
- Khan, A., J. Hughes, D. Valentine, L. Ruis, K. Sachan, A. Radhakrishnan, E. Grefenstette, S. R. Bowman, T. Rocktäschel, and E. Perez (2024). *Debating with More Persuasive LLMs Leads to More Truthful Answers*. ICML 2024 (Best Paper). arXiv: 2402.06782. URL: <https://arxiv.org/abs/2402.06782>.
- Kim, M. J. et al. (2024a). *OpenVLA: An Open-Source Vision-Language-Action Model*. arXiv / CoRL 2024. arXiv: 2406.09246. URL: <https://arxiv.org/abs/2406.09246>.
- Kim, Y. et al. (2024b). *MDAgents: An Adaptive Collaboration of LLMs for Medical Decision-Making*. NeurIPS 2024. arXiv: 2404.15155. URL: <https://arxiv.org/abs/2404.15155>.
- Kimi Team (2025). *Kimi K2: Open Agentic Intelligence*. arXiv: 2507.20534. URL: <https://arxiv.org/abs/2507.20534>.
- Kojima, T., S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa (2022). *Large Language Models are Zero-Shot Reasoners*. NeurIPS 2022. arXiv: 2205.11916. URL: <https://arxiv.org/abs/2205.11916>.
- Kumar, A., V. Zhuang, R. Agarwal, Y. Su, J. D. Co-Reyes, A. Singh, K. Baumli, S. Iqbal, C. Bishop, R. Roelofs, L. M. Zhang, K. McKinney, D. Shrivastava, C. Paduraru, G. Tucker, D. Precup, F. Behbahani, and A. Faust (2024). *Training Language Models to Self-Correct via Reinforcement Learning*. ICLR 2025. arXiv: 2409.12917. URL: <https://arxiv.org/abs/2409.12917>.
- Kuntz, T., A. Duzan, H. Zhao, et al. (2025). *OS-Harm: A Benchmark for Measuring Safety of Computer Use Agents*. arXiv: 2506.14866. URL: <https://arxiv.org/abs/2506.14866>.
- Lee, J., S. Hong, S. Lee, J. Seo, J. Son, S. Eo, C. Park, H. Park, H. Moon, and H. Lim (2026a). *DART: Draft-Agreement Routing for Training-Free Adaptive Thinking Budgets in Hybrid Reasoning Models*. arXiv: 2606.23181. URL: <https://arxiv.org/abs/2606.23181>.
- Lee, J., S. Hong, H. Moon, and H. Lim (2025). “Cross-Lingual Optimization for Language Transfer in Large Language Models”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv: 2505.14297. URL: <https://arxiv.org/abs/2505.14297>.
- Lee, J., H. Moon, S. Lee, C. Park, S. Eo, H. Ko, J. Seo, S. Lee, and H. Lim (2024). “Length-aware Byte Pair Encoding for Mitigating Over-segmentation in Korean Machine Translation”. In: *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 2287–2303. URL: <https://aclanthology.org/2024.findings-acl.135/>.
- Lee, J., C. Park, and H. Lim (2026b). *To Isolate or to Score? Model-Adaptive Assessment for Cost-Efficient Multi-Agent RAG*. arXiv: 2606.25191. URL: <https://arxiv.org/abs/2606.25191>.
- Lee, J., M. Shim, S. Son, C. Park, Y. Kim, and H. Lim (2022). *There Is No Rose Without a Thorn: Finding Weaknesses on BlenderBot 2.0 in Terms of Model, Data and User-Centric Approach*. arXiv: 2201.03239. URL: <https://arxiv.org/abs/2201.03239>.
- Li, G., H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem (2023a). *CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society*. NeurIPS 2023. arXiv: 2303.17760. URL: <https://arxiv.org/abs/2303.17760>.
- Li, J., Y. Lai, W. Li, J. Ren, M. Zhang, X. Kang, S. Wang, P. Li, Y.-Q. Zhang, W. Ma, et al. (2024a). *Agent Hospital: A Simulacrum of Hospital with Evolvable Medical Agents*. arXiv. arXiv: 2405.02957. URL: <https://arxiv.org/abs/2405.02957>.
- Li, J., Q. Zhang, Y. Yu, Q. Fu, and D. Ye (2024b). *More Agents Is All You Need*. arXiv / TMLR. arXiv: 2402.05120. URL: <https://arxiv.org/abs/2402.05120>.

- Li, M., Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li (2023b). *API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs*. arXiv (EMNLP 2023). arXiv: 2304.08244. URL: <https://arxiv.org/abs/2304.08244>.
- Liang, T., Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, S. Shi, and Z. Tu (2023). *Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate*. EMNLP 2024 (arXiv preprint 2023). arXiv: 2305.19118. URL: <https://arxiv.org/abs/2305.19118>.
- Lightman, H., V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe (2023). *Let’s Verify Step by Step*. ICLR 2024. arXiv: 2305.20050. URL: <https://arxiv.org/abs/2305.20050>.
- Liu, B., X. Li, J. Zhang, et al. (2025). *Advances and Challenges in Foundation Agents: From Brain-Inspired Intelligence to Evolutionary, Collaborative, and Safe Systems*. arXiv: 2504.01990. URL: <https://arxiv.org/abs/2504.01990>.
- Liu, J., K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou (2024a). *Large Language Model-Based Agents for Software Engineering: A Survey*. arXiv. arXiv: 2409.02977. URL: <https://arxiv.org/abs/2409.02977>.
- Liu, L., X. Yang, Y. Shen, B. Hu, Z. Zhang, J. Gu, and G. Zhang (2023a). *Think-in-Memory: Recalling and Post-thinking Enable LLMs with Long-Term Memory*. arXiv preprint. arXiv: 2311.08719. URL: <https://arxiv.org/abs/2311.08719>.
- Liu, W., X. Huang, X. Zeng, X. Hao, S. Yu, D. Li, S. Wang, et al. (2024b). *ToolACE: Winning the Points of LLM Function Calling*. arXiv. arXiv: 2409.00920. URL: <https://arxiv.org/abs/2409.00920>.
- Liu, X., H. Yu, H. Zhang, Y. Xu, et al. (2024c). *AgentBench: Evaluating LLMs as Agents*. ICLR 2024. arXiv: 2308.03688. URL: <https://arxiv.org/abs/2308.03688>.
- Liu, Z., Y. Zhang, P. Li, Y. Liu, and D. Yang (2023b). *A Dynamic LLM-Powered Agent Network for Task-Oriented Agent Collaboration*. arXiv. arXiv: 2310.02170. URL: <https://arxiv.org/abs/2310.02170>.
- Lu, C., C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha (2024). *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. arXiv. arXiv: 2408.06292. URL: <https://arxiv.org/abs/2408.06292>.
- Lu, P., B. Peng, H. Cheng, M. Galley, K.-W. Chang, Y. N. Wu, S.-C. Zhu, and J. Gao (2023). *Chameleon: Plug-and-Play Compositional Reasoning with Large Language Models*. NeurIPS 2023. arXiv: 2304.09842. URL: <https://arxiv.org/abs/2304.09842>.
- Luo, J., W. Zhang, Y. Yuan, Y. Zhao, D. Tao, et al. (2025). *Large Language Model Agent: A Survey on Methodology, Applications and Challenges*. arXiv. arXiv: 2503.21460. URL: <https://arxiv.org/abs/2503.21460>.
- Ma, C., J. Zhang, Z. Zhu, C. Yang, Y. Yang, Y. Jin, Z. Lan, L. Kong, and J. He (2024). *AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents*. NeurIPS 2024. arXiv: 2401.13178. URL: <https://arxiv.org/abs/2401.13178>.
- Madaan, A., N. Tandon, P. Gupta, S. Hallinan, L. Gao, et al. (2023). *SELF-REFINE: Iterative Refinement with Self-Feedback*. NeurIPS 2023. arXiv: 2303.17651. URL: <https://arxiv.org/abs/2303.17651>.
- Maharana, A., D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang (2024). *Evaluating Very Long-Term Conversational Memory of LLM Agents*. ACL 2024. arXiv: 2402.17753. URL: <https://arxiv.org/abs/2402.17753>.
- Masterman, T., S. Besen, M. Sawtell, and A. Chao (2024). *The Landscape of Emerging AI Agent Architectures for Reasoning, Planning, and Tool Calling: A Survey*. arXiv. arXiv: 2404.11584. URL: <https://arxiv.org/abs/2404.11584>.
- Meinke, A., B. Schoen, J. Scheurer, M. Balesni, R. Shah, and M. Hobbhahn (2024). *Frontier Models are Capable of In-context Scheming*. arXiv (Apollo Research). arXiv: 2412.04984. URL: <https://arxiv.org/abs/2412.04984>.

- Merrill, M. A., A. G. Shaw, N. Carlini, et al. (2026). *Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces*. arXiv: 2601.11868. URL: <https://arxiv.org/abs/2601.11868>.
- Mialon, G., C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom (2023). *GAIA: a benchmark for General AI Assistants*. ICLR 2024. arXiv: 2311.12983. URL: <https://arxiv.org/abs/2311.12983>.
- Miserendino, S., M. Wang, T. Patwardhan, and J. Heidecke (2025). *SWE-Lancer: Can Frontier LLMs Earn \$1 Million from Real-World Freelance Software Engineering?* arXiv: 2502.12115. URL: <https://arxiv.org/abs/2502.12115>.
- Mitchell, M., A. Ghosh, A. S. Luccioni, and G. Pistilli (2025). *Fully Autonomous AI Agents Should Not Be Developed*. arXiv: 2502.02649. URL: <https://arxiv.org/abs/2502.02649>.
- Mitchener, L., A. Yiu, B. Chang, et al. (2025). *Kosmos: An AI Scientist for Autonomous Discovery*. arXiv: 2511.02824. URL: <https://arxiv.org/abs/2511.02824>.
- Modarressi, A., A. Imani, M. Fayyaz, and H. Schütze (2023). *RET-LLM: Towards a General Read-Write Memory for Large Language Models*. arXiv preprint. arXiv: 2305.14322. URL: <https://arxiv.org/abs/2305.14322>.
- Mohammadi, M., Y. Li, J. Lo, and W. Yip (2025). *Evaluation and Benchmarking of LLM Agents: A Survey*. KDD 2025. arXiv: 2507.21504. URL: <https://arxiv.org/abs/2507.21504>.
- Nakano, R., J. Hilton, S. Balaji, J. Wu, L. Ouyang, et al. (2021). *WebGPT: Browser-assisted question-answering with human feedback*. arXiv (OpenAI). arXiv: 2112.09332. URL: <https://arxiv.org/abs/2112.09332>.
- Nguyen, D. et al. (2024). *GUI Agents: A Survey*. Findings of ACL 2025. arXiv: 2412.13501. URL: <https://arxiv.org/abs/2412.13501>.
- Ning, L., Z. Liang, Z. Jiang, H. Qu, Y. Ding, W. Fan, X.-y. Wei, S. Lin, H. Liu, P. S. Yu, and Q. Li (2025). *A Survey of WebAgents: Towards Next-Generation AI Agents for Web Automation with Large Foundation Models*. KDD 2025. arXiv: 2503.23350. URL: <https://arxiv.org/abs/2503.23350>.
- Novikov, A., N. Vu, M. Eisenberger, E. Dupont, P.-S. Huang, et al. (2025). *AlphaEvolve: A coding agent for scientific and algorithmic discovery*. arXiv: 2506.13131. URL: <https://arxiv.org/abs/2506.13131>.
- OpenAI (2025). *Introducing Operator*. URL: <https://openai.com/index/introducing-operator/> (visited on 07/08/2026).
- Ouyang, S., J. Yan, I.-H. Hsu, et al. (2025). *ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory*. arXiv: 2509.25140. URL: <https://arxiv.org/abs/2509.25140>.
- Packer, C., S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez (2023). *MemGPT: Towards LLMs as Operating Systems*. COLM 2024. arXiv: 2310.08560. URL: <https://arxiv.org/abs/2310.08560>.
- Parisi, A., Y. Zhao, and N. Fiedel (2022). *TALM: Tool Augmented Language Models*. arXiv: 2205.12255. URL: <https://arxiv.org/abs/2205.12255>.
- Park, J. S., J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein (2023a). *Generative Agents: Interactive Simulacra of Human Behavior*. arXiv preprint; UIST 2023. arXiv: 2304.03442. URL: <https://arxiv.org/abs/2304.03442>.
- Park, P. S., S. Goldstein, A. O’Gara, M. Chen, and D. Hendrycks (2023b). *AI Deception: A Survey of Examples, Risks, and Potential Solutions*. Patterns (Cell Press) / arXiv: 2308.14752. URL: <https://arxiv.org/abs/2308.14752>.
- Patil, S. G., H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez (2025). “The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, pp. 48371–48392. URL: <https://proceedings.mlr.press/v267/patil25a.html>.

- Patil, S. G., T. Zhang, X. Wang, and J. E. Gonzalez (2024). *Gorilla: Large Language Model Connected with Massive APIs*. NeurIPS 2024. arXiv: 2305.15334. URL: <https://arxiv.org/abs/2305.15334>.
- Perez, E., S. Ringer, K. Lukošiušė, et al. (2022). *Discovering Language Model Behaviors with Model-Written Evaluations*. arXiv (Anthropic). arXiv: 2212.09251. URL: <https://arxiv.org/abs/2212.09251>.
- Plaat, A., M. van Duijn, N. van Stein, M. Preuss, P. van der Putten, and K. J. Batenburg (2025). *Agentic Large Language Models, a Survey*. arXiv. arXiv: 2503.23037. URL: <https://arxiv.org/abs/2503.23037>.
- Prasad, A., A. Koller, M. Hartmann, P. Clark, A. Sabharwal, M. Bansal, and T. Khot (2023). *ADaPT: As-Needed Decomposition and Planning with Language Models*. NAACL 2024 Findings. arXiv: 2311.05772. URL: <https://arxiv.org/abs/2311.05772>.
- Qi, Z., X. Liu, I. L. Iong, H. Lai, X. Sun, W. Zhao, Y. Yang, X. Yang, J. Sun, S. Yao, T. Zhang, W. Xu, J. Tang, and Y. Dong (2024). *WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum Reinforcement Learning*. arXiv: 2411.02337. URL: <https://arxiv.org/abs/2411.02337>.
- Qian, C., W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun (2023). *ChatDev: Communicative Agents for Software Development*. ACL 2024 (arXiv preprint 2023). arXiv: 2307.07924. URL: <https://arxiv.org/abs/2307.07924>.
- Qian, C., Z. Xie, Y. Wang, et al. (2024). *Scaling Large Language Model-based Multi-Agent Collaboration*. arXiv: 2406.07155. URL: <https://arxiv.org/abs/2406.07155>.
- Qin, Y. et al. (2025). *UI-TARS: Pioneering Automated GUI Interaction with Native Agents*. arXiv. arXiv: 2501.12326. URL: <https://arxiv.org/abs/2501.12326>.
- Qin, Y., S. Liang, Y. Ye, K. Zhu, L. Yan, et al. (2023). *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs*. arXiv (ICLR 2024). arXiv: 2307.16789. URL: <https://arxiv.org/abs/2307.16789>.
- Qu, C., S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, and J.-R. Wen (2024). *Tool Learning with Large Language Models: A Survey*. arXiv; Frontiers of Computer Science 2024 (DOI: 10.1007/s11704-024-40678-2). arXiv: 2405.17935. URL: <https://arxiv.org/abs/2405.17935>.
- Rawles, C. et al. (2024). *AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents*. arXiv (ICLR 2025 submission). arXiv: 2405.14573. URL: <https://arxiv.org/abs/2405.14573>.
- Ren, S., C. Xie, P. Jian, Z. Ren, C. Leng, and J. Zhang (2025). *Towards Scientific Intelligence: A Survey of LLM-based Scientific Agents*. arXiv. arXiv: 2503.24047. URL: <https://arxiv.org/abs/2503.24047>.
- Ruan, Y., H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto (2024). *Identifying the Risks of LM Agents with an LM-Emulated Sandbox*. ICLR 2024 (Spotlight). arXiv: 2309.15817. URL: <https://arxiv.org/abs/2309.15817>.
- Sapkota, R., K. I. Roumeliotis, and M. Karkee (2025). *AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges*. arXiv. arXiv: 2505.10468. URL: <https://arxiv.org/abs/2505.10468>.
- Schick, T., J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. arXiv preprint; NeurIPS 2023. arXiv: 2302.04761. URL: <https://arxiv.org/abs/2302.04761>.
- Schmidgall, S., R. Ziaei, C. Harris, et al. (2024). *AgentClinic: A Multimodal Agent Benchmark to Evaluate AI in Simulated Clinical Environments*. arXiv: 2405.07960. URL: <https://arxiv.org/abs/2405.07960>.
- Sharma, M., M. Tong, T. Korbak, D. Duvenaud, A. Askill, et al. (2023). *Towards Understanding Sycophancy in Language Models*. arXiv (Anthropic). arXiv: 2310.13548. URL: <https://arxiv.org/abs/2310.13548>.
- Shen, W., C. Li, H. Chen, M. Yan, X. Quan, et al. (2024). *Small LLMs Are Weak Tool Learners: A Multi-LLM Agent*. EMNLP 2024. arXiv: 2401.07324. URL: <https://arxiv.org/abs/2401.07324>.

- Shen, Y., K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang (2023). *HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face*. arXiv preprint; NeurIPS 2023. arXiv: 2303.17580. URL: <https://arxiv.org/abs/2303.17580>.
- Shinn, N., F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv preprint; NeurIPS 2023. arXiv: 2303.11366. URL: <https://arxiv.org/abs/2303.11366>.
- Shridhar, M., X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht (2021). *ALFWorld: Aligning Text and Embodied Environments for Interactive Learning*. ICLR 2021. arXiv: 2010.03768. URL: <https://arxiv.org/abs/2010.03768>.
- Smit, A., P. Duckworth, N. Grinsztajn, T. D. Barrett, and A. Pretorius (2023). *Should we be going MAD? A Look at Multi-Agent Debate Strategies for LLMs*. arXiv preprint (ICML 2024 workshop). arXiv: 2311.17371. URL: <https://arxiv.org/abs/2311.17371>.
- Song, Y., W. Xiong, D. Zhu, W. Wu, H. Qian, et al. (2023). *RestGPT: Connecting Large Language Models with Real-World RESTful APIs*. arXiv. arXiv: 2306.06624. URL: <https://arxiv.org/abs/2306.06624>.
- Stauffer, L., K. Feng, K. Wei, et al. (2026). *The 2025 AI Agent Index: Documenting Technical and Safety Features of Deployed Agentic AI Systems*. ACM FAccT 2026 (arXiv 2026). arXiv: 2602.17753. URL: <https://arxiv.org/abs/2602.17753>.
- Su, H., J. Luo, C. Liu, X. Yang, Y. Zhang, Y. Dong, and J. Zhu (2025). *A Survey on Autonomy-Induced Security Risks in Large Model-Based Agents*. arXiv. arXiv: 2506.23844. URL: <https://arxiv.org/abs/2506.23844>.
- Sumers, T. R., S. Yao, K. Narasimhan, and T. L. Griffiths (2023). *Cognitive Architectures for Language Agents*. arXiv; TMLR (camera-ready v3). arXiv: 2309.02427. URL: <https://arxiv.org/abs/2309.02427>.
- Tang, Q., Z. Deng, H. Lin, X. Han, Q. Liang, B. Cao, and L. Sun (2023). *ToolAlpaca: Generalized Tool Learning for Language Models with 3000 Simulated Cases*. arXiv. arXiv: 2306.05301. URL: <https://arxiv.org/abs/2306.05301>.
- Tian, Y., X. Yang, J. Zhang, Y. Dong, and H. Su (2023). *Evil Geniuses: Delving into the Safety of LLM-based Agents*. arXiv. arXiv: 2311.11855. URL: <https://arxiv.org/abs/2311.11855>.
- Tongyi DeepResearch Team (2025). *Tongyi DeepResearch Technical Report*. arXiv: 2510.24701. URL: <https://arxiv.org/abs/2510.24701>.
- Tran, K.-T., D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O’Sullivan, and H. D. Nguyen (2025). *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*. arXiv. arXiv: 2501.06322. URL: <https://arxiv.org/abs/2501.06322>.
- Tu, T. et al. (2024). *Towards Conversational Diagnostic AI*. arXiv. arXiv: 2401.05654. URL: <https://arxiv.org/abs/2401.05654>.
- Wallace, E., K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel (2024). *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*. arXiv: 2404.13208. URL: <https://arxiv.org/abs/2404.13208>.
- Wang, G., Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar (2023a). *Voyager: An Open-Ended Embodied Agent with Large Language Models*. arXiv preprint; TMLR. arXiv: 2305.16291. URL: <https://arxiv.org/abs/2305.16291>.
- Wang, H., H. Zou, H. Song, et al. (2025a). *UI-TARS-2 Technical Report: Advancing GUI Agent with Multi-Turn Reinforcement Learning*. arXiv: 2509.02544. URL: <https://arxiv.org/abs/2509.02544>.
- Wang, J., Z. Ma, Y. Li, S. Zhang, C. Chen, K. Chen, and X. Le (2024a). *GTA: A Benchmark for General Tool Agents*. NeurIPS 2024. arXiv: 2407.08713. URL: <https://arxiv.org/abs/2407.08713>.
- Wang, J., J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou (2024b). *Mixture-of-Agents Enhances Large Language Model Capabilities*. arXiv preprint (ICLR 2025). arXiv: 2406.04692. URL: <https://arxiv.org/abs/2406.04692>.

- Wang, J., H. Xu, J. Ye, M. Yan, W. Shen, J. Zhang, F. Huang, and J. Sang (2024c). *Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception*. arXiv. arXiv: 2401.16158. URL: <https://arxiv.org/abs/2401.16158>.
- Wang, K. et al. (2025b). *A Comprehensive Survey in LLM(-Agent) Full Stack Safety: Data, Training and Deployment*. arXiv. arXiv: 2504.15585. URL: <https://arxiv.org/abs/2504.15585>.
- Wang, L., C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen (2023b). *A Survey on Large Language Model based Autonomous Agents*. arXiv preprint; also published in *Frontiers of Computer Science* 2024 (DOI: 10.1007/s11704-024-40231-1). arXiv: 2308.11432. URL: <https://arxiv.org/abs/2308.11432>.
- Wang, L., W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim (2023c). *Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models*. ACL 2023. arXiv: 2305.04091. URL: <https://arxiv.org/abs/2305.04091>.
- Wang, W., L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao, and F. Wei (2023d). *Augmenting Language Models with Long-Term Memory*. NeurIPS 2023. arXiv: 2306.07174. URL: <https://arxiv.org/abs/2306.07174>.
- Wang, W., Z. Ma, Z. Wang, C. Wu, J. Ji, W. Chen, X. Li, and Y. Yuan (2025c). *A Survey of LLM-based Agents in Medicine: How far are we from Baymax?* ACL 2025 Findings. arXiv: 2502.11211. URL: <https://arxiv.org/abs/2502.11211>.
- Wang, X. et al. (2024d). *OpenHands: An Open Platform for AI Software Developers as Generalist Agents*. ICLR 2025 / arXiv. arXiv: 2407.16741. URL: <https://arxiv.org/abs/2407.16741>.
- Wang, X., Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji (2024e). *Executable Code Actions Elicit Better LLM Agents*. ICML 2024. arXiv: 2402.01030. URL: <https://arxiv.org/abs/2402.01030>.
- Wang, X., J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. ICLR 2023. arXiv: 2203.11171. URL: <https://arxiv.org/abs/2203.11171>.
- Wang, Y., D. Xue, S. Zhang, and S. Qian (2024f). *BadAgent: Inserting and Activating Backdoor Attacks in LLM Agents*. ACL 2024. arXiv: 2406.03007. URL: <https://arxiv.org/abs/2406.03007>.
- Wang, Y. and X. Chen (2025). *MIRIX: Multi-Agent Memory System for LLM-Based Agents*. arXiv preprint. arXiv: 2507.07957. URL: <https://arxiv.org/abs/2507.07957>.
- Wang, Z., S. Mao, W. Wu, T. Ge, F. Wei, and H. Ji (2023e). *Unleashing the Emergent Cognitive Synergy in Large Language Models: A Task-Solving Agent through Multi-Persona Self-Collaboration*. NAACL 2024 (arXiv preprint 2023). arXiv: 2307.05300. URL: <https://arxiv.org/abs/2307.05300>.
- Wang, Z., Z. Cheng, H. Zhu, D. Fried, and G. Neubig (2024g). *What Are Tools Anyway? A Survey from the Language Model Perspective*. COLM 2024. arXiv: 2403.15452. URL: <https://arxiv.org/abs/2403.15452>.
- Wang, Z., S. Cai, G. Chen, A. Liu, X. Ma, and Y. Liang (2023f). *Describe, Explain, Plan and Select: Interactive Planning with Large Language Models Enables Open-World Multi-Task Agents*. NeurIPS 2023. arXiv: 2302.01560. URL: <https://arxiv.org/abs/2302.01560>.
- Wei, J., X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022. arXiv: 2201.11903. URL: <https://arxiv.org/abs/2201.11903>.
- Wu, D., H. Wang, W. Yu, et al. (2024a). *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*. ICLR 2025 (arXiv 2024). arXiv: 2410.10813. URL: <https://arxiv.org/abs/2410.10813>.
- Wu, Q., G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang (2023a). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv. arXiv: 2308.08155. URL: <https://arxiv.org/abs/2308.08155>.

- Wu, Y., S. Liang, C. Zhang, Y. Wang, Y. Zhang, H. Guo, R. Tang, and Y. Liu (2025). *From Human Memory to AI Memory: A Survey on Memory Mechanisms in the Era of LLMs*. arXiv preprint. arXiv: 2504.15965. URL: <https://arxiv.org/abs/2504.15965>.
- Wu, Y., X. Tang, T. M. Mitchell, and Y. Li (2023b). *SmartPlay: A Benchmark for LLMs as Intelligent Agents*. ICLR 2024. arXiv: 2310.01557. URL: <https://arxiv.org/abs/2310.01557>.
- Wu, Z., C. Han, Z. Ding, Z. Weng, Z. Liu, S. Yao, T. Yu, and L. Kong (2024b). *OS-Copilot: Towards Generalist Computer Agents with Self-Improvement*. arXiv. arXiv: 2402.07456. URL: <https://arxiv.org/abs/2402.07456>.
- Xi, Z., W. Chen, X. Guo, W. He, Y. Ding, et al. (2023). *The Rise and Potential of Large Language Model Based Agents: A Survey*. arXiv. arXiv: 2309.07864. URL: <https://arxiv.org/abs/2309.07864>.
- Xi, Z., Y. Ding, W. Chen, B. Hong, H. Guo, J. Wang, D. Yang, C. Liao, X. Guo, W. He, S. Gao, L. Chen, R. Zheng, Y. Zou, T. Gui, Q. Zhang, X. Qiu, X. Huang, Z. Wu, and Y.-G. Jiang (2024). *AgentGym: Evolving Large Language Model-based Agents across Diverse Environments*. arXiv: 2406.04151. URL: <https://arxiv.org/abs/2406.04151>.
- Xia, C. S., Y. Deng, S. Dunn, and L. Zhang (2024). *Agentless: Demystifying LLM-based Software Engineering Agents*. FSE 2025 / arXiv. arXiv: 2407.01489. URL: <https://arxiv.org/abs/2407.01489>.
- Xiao, Y., E. Sun, D. Luo, and W. Wang (2024). *TradingAgents: Multi-Agents LLM Financial Trading Framework*. arXiv. arXiv: 2412.20138. URL: <https://arxiv.org/abs/2412.20138>.
- Xie, J., K. Zhang, J. Chen, T. Zhu, R. Lou, Y. Tian, Y. Xiao, and Y. Su (2024a). *TravelPlanner: A Benchmark for Real-World Planning with Language Agents*. ICML 2024. arXiv: 2402.01622. URL: <https://arxiv.org/abs/2402.01622>.
- Xie, T., D. Zhang, J. Chen, et al. (2024b). *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*. NeurIPS 2024. arXiv: 2404.07972. URL: <https://arxiv.org/abs/2404.07972>.
- Xu, B., Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu (2023). *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*. arXiv. arXiv: 2305.18323. URL: <https://arxiv.org/abs/2305.18323>.
- Xu, W., Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang (2025). *A-MEM: Agentic Memory for LLM Agents*. arXiv preprint. arXiv: 2502.12110. URL: <https://arxiv.org/abs/2502.12110>.
- Yamada, Y., R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha (2025). *The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search*. arXiv. arXiv: 2504.08066. URL: <https://arxiv.org/abs/2504.08066>.
- Yan, B., Z. Zhou, L. Zhang, L. Zhang, Z. Zhou, D. Miao, Z. Li, C. Li, and X. Zhang (2025). *Beyond Self-Talk: A Communication-Centric Survey of LLM-Based Multi-Agent Systems*. arXiv (Frontiers of Computer Science). arXiv: 2502.14321. URL: <https://arxiv.org/abs/2502.14321>.
- Yang, H., X.-Y. Liu, and C. D. Wang (2023a). *FinGPT: Open-Source Financial Large Language Models*. IJCAI 2023 FinLLM Workshop / arXiv. arXiv: 2306.06031. URL: <https://arxiv.org/abs/2306.06031>.
- Yang, H., S. Yue, and Y. He (2023b). *Auto-GPT for Online Decision Making: Benchmarks and Additional Opinions*. arXiv. arXiv: 2306.02224. URL: <https://arxiv.org/abs/2306.02224>.
- Yang, J., C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press (2024a). *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024. arXiv: 2405.15793. URL: <https://arxiv.org/abs/2405.15793>.
- Yang, J., A. Prabhakar, K. Narasimhan, and S. Yao (2023c). *InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback*. NeurIPS 2023. arXiv: 2306.14898. URL: <https://arxiv.org/abs/2306.14898>.

- Yang, L., Z. Yu, T. Zhang, S. Cao, M. Xu, W. Zhang, J. E. Gonzalez, and B. Cui (2024b). *Buffer of Thoughts: Thought-Augmented Reasoning with Large Language Models*. NeurIPS 2024. arXiv: 2406.04271. URL: <https://arxiv.org/abs/2406.04271>.
- Yang, R., L. Song, Y. Li, S. Zhao, Y. Ge, X. Li, and Y. Shan (2023d). *GPT4Tools: Teaching Large Language Model to Use Tools via Self-instruction*. arXiv (NeurIPS 2023). arXiv: 2305.18752. URL: <https://arxiv.org/abs/2305.18752>.
- Yang, Y., H. Chai, Y. Song, et al. (2025). *A Survey of AI Agent Protocols*. arXiv: 2504.16736. URL: <https://arxiv.org/abs/2504.16736>.
- Yao, S., H. Chen, J. Yang, and K. Narasimhan (2022a). *WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents*. NeurIPS 2022. arXiv: 2207.01206. URL: <https://arxiv.org/abs/2207.01206>.
- Yao, S., N. Shinn, P. Razavi, and K. Narasimhan (2024). τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv. arXiv: 2406.12045. URL: <https://arxiv.org/abs/2406.12045>.
- Yao, S., D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023. arXiv: 2305.10601. URL: <https://arxiv.org/abs/2305.10601>.
- Yao, S., J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao (2022b). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv preprint; ICLR 2023 (camera-ready version noted on arXiv page). arXiv: 2210.03629. URL: <https://arxiv.org/abs/2210.03629>.
- Yehudai, A., L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer (2025). *Survey on Evaluation of LLM-based Agents*. arXiv. arXiv: 2503.16416. URL: <https://arxiv.org/abs/2503.16416>.
- Yin, S., X. Pang, Y. Ding, M. Chen, Y. Bi, Y. Xiong, W. Huang, Z. Xiang, J. Shao, and S. Chen (2024). *SafeAgentBench: A Benchmark for Safe Task Planning of Embodied LLM Agents*. arXiv. arXiv: 2412.13178. URL: <https://arxiv.org/abs/2412.13178>.
- Yin, Z., Q. Sun, C. Chang, Q. Guo, J. Dai, X. Huang, and X. Qiu (2023). *Exchange-of-Thought: Enhancing Large Language Model Capabilities through Cross-Model Communication*. EMNLP 2023. arXiv: 2312.01823. URL: <https://arxiv.org/abs/2312.01823>.
- Yu, M., F. Meng, X. Zhou, S. Wang, J. Mao, L. Pang, T. Chen, K. Wang, X. Li, Y. Zhang, B. An, and Q. Wen (2025). *A Survey on Trustworthy LLM Agents: Threats and Countermeasures*. arXiv. arXiv: 2503.09648. URL: <https://arxiv.org/abs/2503.09648>.
- Yu, Y., H. Li, Z. Chen, Y. Jiang, Y. Li, D. Zhang, R. Liu, J. W. Suchow, and K. Khashanah (2023). *FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design*. AAAI Symposium Series 2024 / arXiv. arXiv: 2311.13743. URL: <https://arxiv.org/abs/2311.13743>.
- Yuan, T., Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, and G. Liu (2024). *R-Judge: Benchmarking Safety Risk Awareness for LLM Agents*. EMNLP 2024 Findings. arXiv: 2401.10019. URL: <https://arxiv.org/abs/2401.10019>.
- Zelikman, E., Y. Wu, J. Mu, and N. D. Goodman (2022). *STaR: Bootstrapping Reasoning With Reasoning*. NeurIPS 2022. arXiv: 2203.14465. URL: <https://arxiv.org/abs/2203.14465>.
- Zeng, A., M. Liu, R. Lu, B. Wang, X. Liu, Y. Dong, and J. Tang (2023). *AgentTuning: Enabling Generalized Agent Abilities for LLMs*. arXiv: 2310.12823. URL: <https://arxiv.org/abs/2310.12823>.
- Zeng, R., J. Fang, S. Liu, and Z. Meng (2024). *On the Structural Memory of LLM Agents*. arXiv preprint. arXiv: 2412.15266. URL: <https://arxiv.org/abs/2412.15266>.
- Zhan, Q., Z. Liang, Z. Ying, and D. Kang (2024). *InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents*. ACL 2024 Findings. arXiv: 2403.02691. URL: <https://arxiv.org/abs/2403.02691>.

- Zhang, C., S. He, J. Qian, B. Li, L. Li, S. Qin, Y. Kang, M. Ma, G. Liu, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang (2024a). *Large Language Model-Brained GUI Agents: A Survey*. arXiv. arXiv: 2411.18279. URL: <https://arxiv.org/abs/2411.18279>.
- Zhang, C., Z. Yang, J. Liu, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu (2023a). *AppAgent: Multimodal Agents as Smartphone Users*. arXiv (later ACM CHI 2025). arXiv: 2312.13771. URL: <https://arxiv.org/abs/2312.13771>.
- Zhang, G., H. Geng, X. Yu, et al. (2025). *The Landscape of Agentic Reinforcement Learning for LLMs: A Survey*. arXiv: 2509.02547. URL: <https://arxiv.org/abs/2509.02547>.
- Zhang, H., J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang (2024b). *Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents*. arXiv. arXiv: 2410.02644. URL: <https://arxiv.org/abs/2410.02644>.
- Zhang, H., W. Du, J. Shan, Q. Zhou, Y. Du, J. B. Tenenbaum, T. Shu, and C. Gan (2023b). *Building Cooperative Embodied Agents Modularly with Large Language Models*. ICLR 2024 (arXiv preprint 2023). arXiv: 2307.02485. URL: <https://arxiv.org/abs/2307.02485>.
- Zhang, J., T. Lan, M. Zhu, Z. Liu, T. Hoang, et al. (2024c). *xLAM: A Family of Large Action Models to Empower AI Agent Systems*. arXiv. arXiv: 2409.03215. URL: <https://arxiv.org/abs/2409.03215>.
- Zhang, J., J. Xiang, Z. Yu, et al. (2024d). *AFlow: Automating Agentic Workflow Generation*. ICLR 2025 (arXiv 2024). arXiv: 2410.10762. URL: <https://arxiv.org/abs/2410.10762>.
- Zhang, Y., H. Ruan, Z. Fan, and A. Roychoudhury (2024e). *AutoCodeRover: Autonomous Program Improvement*. ISSA 2024 / arXiv. arXiv: 2404.05427. URL: <https://arxiv.org/abs/2404.05427>.
- Zhang, Z., X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen (2024f). *A Survey on the Memory Mechanism of Large Language Model based Agents*. arXiv. arXiv: 2404.13501. URL: <https://arxiv.org/abs/2404.13501>.
- Zhang, Z., Y. Yao, A. Zhang, X. Tang, X. Ma, Z. He, Y. Wang, M. Gerstein, R. Wang, G. Liu, and H. Zhao (2023c). *Igniting Language Intelligence: The Hitchhiker’s Guide From Chain-of-Thought Reasoning to Language Agents*. arXiv (ACM Computing Surveys, as claimed). arXiv: 2311.11797. URL: <https://arxiv.org/abs/2311.11797>.
- Zhao, A., D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang (2023). *ExpeL: LLM Agents Are Experiential Learners*. AAAI 2024. arXiv: 2308.10144. URL: <https://arxiv.org/abs/2308.10144>.
- Zheng, B., B. Gou, J. Kil, H. Sun, and Y. Su (2024). *GPT-4V(ision) is a Generalist Web Agent, if Grounded*. ICML 2024. arXiv: 2401.01614. URL: <https://arxiv.org/abs/2401.01614>.
- Zheng, L., R. Wang, X. Wang, and B. An (2023). *Synapse: Trajectory-as-Exemplar Prompting with Memory for Computer Control*. ICLR 2024. arXiv: 2306.07863. URL: <https://arxiv.org/abs/2306.07863>.
- Zhong, W., L. Guo, Q. Gao, H. Ye, and Y. Wang (2023). *MemoryBank: Enhancing Large Language Models with Long-Term Memory*. AAAI 2024. arXiv: 2305.10250. URL: <https://arxiv.org/abs/2305.10250>.
- Zhou, A., K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang (2024a). *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*. ICML 2024. arXiv: 2310.04406. URL: <https://arxiv.org/abs/2310.04406>.
- Zhou, D., N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le, and E. Chi (2022). *Least-to-Most Prompting Enables Complex Reasoning in Large Language Models*. ICLR 2023. arXiv: 2205.10625. URL: <https://arxiv.org/abs/2205.10625>.
- Zhou, P., J. Pujara, X. Ren, X. Chen, H.-T. Cheng, Q. V. Le, E. H. Chi, D. Zhou, S. Mishra, and H. S. Zheng (2024b). *Self-Discover: Large Language Models Self-Compose Reasoning Structures*. NeurIPS 2024. arXiv: 2402.03620. URL: <https://arxiv.org/abs/2402.03620>.

- Zhou, S., F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig (2024c). *WebArena: A Realistic Web Environment for Building Autonomous Agents*. ICLR 2024. arXiv: 2307.13854. URL: <https://arxiv.org/abs/2307.13854>.
- Zhuge, M., W. Wang, L. Kirsch, et al. (2024a). *Language Agents as Optimizable Graphs*. ICML 2024 (arXiv 2024). arXiv: 2402.16823. URL: <https://arxiv.org/abs/2402.16823>.
- Zhuge, M., C. Zhao, D. Ashley, et al. (2024b). *Agent-as-a-Judge: Evaluate Agents with Agents*. arXiv: 2410.10934. URL: <https://arxiv.org/abs/2410.10934>.
- Zou, H. P., W.-C. Huang, Y. Wu, J. Guo, Y. Chen, et al. (2025). *LLM-Based Human-Agent Collaboration and Interaction Systems: A Survey*. arXiv. arXiv: 2505.00753. URL: <https://arxiv.org/abs/2505.00753>.